

SAMSUNG INNOVATION CAMPUS

Khóa học Trí tuệ Nhân tạo (AI Course)

CAPSTONE PROJECT

FINAL REPORT

ỨNG DỤNG HỌC MÁY XÂY DỰNG HỆ THỐNG DỰ BÁO VÀ HỖ TRỢ RA QUYẾT ĐỊNH KINH DOANH TRÊN NỀN TẢNG WEB

(Dựa trên bộ dữ liệu *Favorita Grocery Sales Forecasting* – Kaggle)

Kết quả Kaggle Leaderboard (Chỉ số NWRMSLE):

Global Ensemble: 0.64438 (Public) | **0.63542 (Private – Best Score)**

Local Models: 0.65307 (Public) | 0.64207 (Private)

Nhóm thực hiện: Team 1

Thành viên: Phạm Hoàng An Nguyễn Thế Công

Mã nguồn (Repo): <https://github.com/RikkaNg/SIC-AI-PROJECT->

Thời gian: Tháng 08/2026

© 2023 SAMSUNG. Bảo lưu mọi quyền.

Văn phòng Trách nhiệm Xã hội Doanh nghiệp Samsung Electronics nắm giữ bản quyền của tài liệu này.

Tài liệu này là tài sản sở hữu trí tuệ được bảo vệ bởi luật bản quyền, nghiêm cấm sao chép và tái bản khi chưa có sự cho phép.

Để sử dụng tài liệu này ngoài chương trình đào tạo của Samsung Innovation Campus, cần phải có sự đồng ý bằng văn bản của chủ sở hữu bản quyền.

Together for Tomorrow!
Enabling People

Education for Future Generations

Mục lục

Tóm tắt	2
Đặt vấn đề	3
1 CƠ SỞ LÝ THUYẾT	5
1.1 Tổng quan về học máy (Machine Learning)	5
1.1.1 Khái niệm	5
1.1.2 Phân loại các bài toán học máy	5
1.1.3 Bài toán hồi quy chuỗi thời gian	6
1.2 Bài toán dự báo chuỗi thời gian (Time Series Forecasting)	6
1.2.1 Định nghĩa	6
1.2.2 Các thành phần của chuỗi thời gian	6
1.2.3 Các phương pháp dự báo truyền thống	7
1.2.4 Học máy trong dự báo chuỗi thời gian	7
1.3 Các mô hình học máy sử dụng trong dự báo	8
1.3.1 Mô hình cây tăng cường Gradient (GBDT)	8
1.3.2 Thuật toán phân cụm (K-Means)	9
1.3.3 Mô hình hóa đa cấp: đánh đổi độ lệch – phương sai	9
1.3.4 Kiến trúc Transformer và LLM Agent	10
1.3.5 Chỉ số đánh giá mô hình dự báo	10
1.4 Nền tảng web và kiến trúc hệ thống hỗ trợ ra quyết định	13
1.4.1 Hệ thống hỗ trợ ra quyết định (DSS)	13
1.4.2 Kiến trúc ứng dụng web hiện đại	13
1.5 Tổng quan tập dữ liệu Favorita Grocery Sales Forecasting	13
2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	15
2.1 Phân tích yêu cầu hệ thống	15
2.1.1 Khảo sát bài toán thực tế	15
2.1.2 Xác định đối tượng người dùng (User Roles & Personas)	15
2.1.3 Yêu cầu kỹ thuật và ràng buộc vận hành	16
2.2 Kiến trúc tổng thể hệ thống	16

2.3	Thiết kế pipeline dữ liệu và huấn luyện mô hình	17
2.4	Thiết kế trợ lý AI thông minh (LLM Agent)	19
2.4.1	Bộ công cụ và toán học chuỗi cung ứng	19
2.4.2	Ràng buộc an toàn của tác tử	20
2.5	Thiết kế cơ sở dữ liệu	20
2.5.1	Mô hình thực thể – quan hệ (ERD)	20
2.5.2	Mô tả các thực thể và thuộc tính	20
2.5.3	Tối ưu hóa cơ sở dữ liệu	21
2.6	Thiết kế giao diện người dùng (UI/UX)	21
2.6.1	Sơ đồ luồng màn hình	21
2.6.2	Thiết kế dashboard trực quan hóa dự báo	22
2.6.3	Thiết kế chức năng hỗ trợ ra quyết định	22
2.7	Thiết kế bảo mật và phân quyền	22
2.7.1	Cơ chế xác thực	22
2.7.2	Phân quyền theo vai trò (RBAC) và theo tenant	23
2.7.3	Đảm bảo cách ly dữ liệu giữa các tenant	23
3	HUẤN LUYỆN MÔ HÌNH VÀ TÍCH HỢP HỆ THỐNG	24
3.1	Môi trường và công cụ huấn luyện	24
3.2	Nạp và làm sạch dữ liệu	24
3.2.1	Tối ưu hóa việc nạp dữ liệu thô (Streaming Ingestion)	24
3.2.2	Làm sạch và Xử lý giá trị bất thường (Data Cleaning)	25
3.2.3	Mô phỏng dữ liệu tồn kho (Inventory Simulation)	26
3.2.4	Kỹ thuật ghi vào Cơ sở dữ liệu (Database Loading)	26
3.3	Feature Engineering (Kỹ thuật đặc trưng)	27
3.3.1	Tạo lập Đặc trưng chuỗi thời gian (Pandas Deterministic Features)	27
3.3.2	Kỹ thuật Target Encoding và Tương tác Cụm cửa hàng (Cluster Features)	28
3.3.3	Mã hóa Biến Phân loại và Xử lý Thiếu dữ liệu (Scikit-learn Pipeline)	28
3.3.4	Biến đổi Biến Mục tiêu (Target Transformation)	28
3.4	Huấn luyện Mô hình và Tích hợp Hệ thống	31
3.4.1	Huấn luyện Global Ensemble (LightGBM + CatBoost)	31
3.4.2	Huấn luyện 33 Local Models	32
3.4.3	Dự báo đệ quy 16 ngày và Smart Routing	32

3.4.4	Chuẩn bị submission Kaggle	33
3.5	Khởi tạo cơ sở dữ liệu và tích hợp triển khai	33
3.5.1	Script khởi tạo	33
3.5.2	Docker Compose deployment	33
3.5.3	Vận hành	34
4	KẾT QUẢ, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	35
4.1	Kết quả huấn luyện mô hình (Validation nội bộ)	35
4.1.1	Global Ensemble — Kiểm định Walk-Forward	35
4.1.2	Ảnh hưởng của điểm số đến các quyết định sau huấn luyện	36
4.1.3	Local Models — 33 ngành hàng	38
4.1.4	Phân tích kết quả validation	39
4.2	Kết quả trên Kaggle Leaderboard	39
4.2.1	Kết quả submission thực tế	39
4.2.2	Phân tích kết quả Kaggle	39
4.2.3	So sánh với baseline và top solution	41
4.2.4	Đánh giá vị trí trên leaderboard	41
4.3	Nhật ký thí nghiệm và bằng chứng vận hành	42
4.3.1	Timeline thí nghiệm	42
4.3.2	Kết quả baseline so với thí nghiệm	42
4.3.3	Bộ sáu chỉ số của bản thí nghiệm	43
4.3.4	Sinh submission và kiểm thử	43
4.3.5	Tái tạo các log	44
4.4	Kiểm thử API Endpoints	44
4.5	Kiểm thử bảo mật	44
4.6	Đánh giá Trợ lý AI (LLM Agent)	45
4.6.1	Chất lượng hội thoại	45
4.6.2	Ví dụ hội thoại thực tế	45
4.7	Đánh giá hiệu năng tổng thể	46
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
5.1	Tổng kết kết quả đạt được	47
5.1.1	Kết quả về mô hình học máy	47
5.1.2	Kết quả về hệ thống web	47

5.2	Hạn chế của hệ thống	47
5.3	Hướng phát triển tương lai	48
5.3.1	Ngắn hạn (3–6 tháng)	48
5.3.2	Trung hạn (6–12 tháng)	48
5.3.3	Dài hạn (1–2 năm)	48
5.4	Bài học kinh nghiệm	48
Đánh giá và Nhận xét		50
Tài liệu tham khảo		51

Tóm tắt

Báo cáo này trình bày quá trình nghiên cứu, thiết kế và triển khai một hệ thống dự báo doanh số bán lẻ và hỗ trợ ra quyết định kinh doanh tích hợp Trí tuệ Nhân tạo trên nền tảng web. Dự án sử dụng bộ dữ liệu **Favorita Grocery Sales Forecasting** (Kaggle) gồm 59 triệu dòng dữ liệu lịch sử từ 54 cửa hàng và 4.100 sản phẩm của chuỗi siêu thị Corporación Favorita (Ecuador).

Hệ thống được xây dựng theo kiến trúc microservices với ba thành phần chính: (1) **Backend Gateway** sử dụng FastAPI xử lý logic nghiệp vụ và bảo mật; (2) **ML Service** cung cấp API dự báo doanh số với chiến lược Smart Routing kết hợp 33 Local LightGBM Models chuyên biệt theo ngành hàng và Global Ensemble (LightGBM + CatBoost) làm tuyến dự phòng; (3) **Frontend** React/Vite cung cấp giao diện dashboard trực quan và chatbot AI.

Điểm đột phá của đề tài là tích hợp **Trợ lý AI thông minh** (LLM Agent) sử dụng mô hình Qwen 3.6-27B trên hạ tầng Groq Cloud LPU với cơ chế **Function Calling**, cho phép người quản lý đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận phân tích, đề xuất nhập hàng tự động dựa trên dữ liệu thực tế từ cơ sở dữ liệu.

Kết quả trên Kaggle Leaderboard: Hệ thống đã submit hai phương án dự báo lên cuộc thi Kaggle Favorita Grocery Sales Forecasting với chỉ số NWRMSLE (Normalized Weighted RMSLE — thấp hơn là tốt hơn):

- **Global Ensemble (Best Score):** Public 0.64438 / Private **0.63542**;
- **Local Models:** Public 0.65307 / Private 0.64207.

Kết quả xác nhận Global Ensemble vượt trội so với Local Models thuần túy (cải thiện 0.0087 điểm public, 0.0067 điểm private), đồng thời private score tốt hơn public score trên cả hai submission cho thấy mô hình có khả năng tổng quát hóa tốt, không overfit.

Toàn bộ hệ thống được đóng gói bằng Docker Compose, hỗ trợ triển khai một lệnh duy nhất, với bảo mật đa tầng bao gồm JWT authentication, Role-Based Access Control (RBAC) và Row-Level Isolation theo phạm vi cửa hàng.

Từ khóa: Học máy, Dự báo chuỗi thời gian, LightGBM, CatBoost, LLM Agent, Function Calling, Groq Cloud, Retail Analytics, Decision Support System, Microservices.

Đặt vấn đề

Bối cảnh đề tài

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ ở hầu hết các ngành nghề, lĩnh vực bán lẻ (retail) là một trong những lĩnh vực chịu áp lực lớn nhất về việc tối ưu hóa vận hành và ra quyết định dựa trên dữ liệu. Các doanh nghiệp bán lẻ, từ chuỗi siêu thị lớn đến cửa hàng vừa và nhỏ, đều phải đối mặt với bài toán cốt lõi: **dự báo chính xác nhu cầu tiêu thụ của từng mặt hàng tại từng cửa hàng theo thời gian**.

Việc dự báo sai lệch dẫn tới hai hệ quả tiêu cực song song:

- **Tồn kho dư thừa** gây lãng phí chi phí lưu kho, vốn ứ đọng, hàng hóa hết hạn (đặc biệt với thực phẩm tươi sống);
- **Thiếu hụt hàng hóa** gây mất doanh thu, giảm trải nghiệm khách hàng và mất thị phần vào tay đối thủ.

Bài toán dự báo doanh số bán lẻ vốn phức tạp bởi chịu ảnh hưởng của nhiều yếu tố đan xen: tính mùa vụ, ngày lễ và sự kiện đặc biệt, chương trình khuyến mãi, biến động giá dầu (đặc biệt tại các quốc gia phụ thuộc xuất khẩu dầu mỏ như Ecuador), đặc điểm vùng miền, loại cửa hàng và cả những cú sốc bất thường. Các phương pháp truyền thống (ARIMA hay kinh nghiệm người quản lý) khó nắm bắt các quan hệ phi tuyến và tương tác phức tạp giữa các yếu tố này ở quy mô hàng nghìn sản phẩm, hàng chục cửa hàng — càng khó hơn với doanh nghiệp vừa và nhỏ không có đủ nguồn lực tự xây dựng hệ thống riêng.

Tính cấp thiết

Trong quản trị chuỗi bán lẻ quy mô lớn, việc kiểm soát tồn kho đối mặt áp lực kép từ rủi ro cháy hàng (*stockout*) gây thiệt hại doanh thu lẫn nguy cơ đọng vốn, hao hụt do tồn kho dư thừa (*overstock*). Các phương pháp thống kê cổ điển hay mô hình học máy đơn lẻ bộc lộ hạn chế khi xử lý các yếu tố ngoại sinh phi tuyến (lịch khuyến mãi, sự kiện ngày lễ, giá dầu thô, sự phân hóa theo cụm cửa hàng), đồng thời tạo độ trễ lớn giữa kết quả dự báo và hành động thực thi vì người quản lý phải tính tay các tham số chuỗi cung ứng như Điểm đặt hàng lại (ROP), Tồn kho an toàn (Safety Stock), Lead Time.

Do đó việc nghiên cứu phát triển một hệ thống kết hợp Machine Learning đa cấp (33 Local Models LightGBM chuyên biệt hóa cùng Global Fallback Ensemble) để dự báo nhu cầu, tích hợp Trí tuệ nhân tạo tác tử (Agentic LLM Qwen 3.6 trên hạ tầng Groq Cloud LPU)

qua cơ chế Function Calling nhằm tự động hóa phân tích dữ liệu và đề xuất phương án nhập hàng theo thời gian thực là yêu cầu có tính cấp thiết cao, giúp doanh nghiệp chuyển từ quản trị phản ứng thụ động sang tự động hóa ra quyết định tối ưu.

Mục tiêu nghiên cứu

1. Xây dựng hệ thống dự báo doanh số bán lẻ ngắn hạn (16 ngày) ở cấp độ SKU × cửa hàng, kiểm chứng chất lượng qua submission lên Kaggle Favorita với chỉ số NWRMSLE;
2. Thiết kế kiến trúc microservices đảm bảo hiệu năng API < 1 giây và khả năng mở rộng ngang;
3. Phát triển trợ lý AI thông hiểu ngôn ngữ tự nhiên, tự động gọi công cụ phân tích dữ liệu và đề xuất hành động;
4. Đảm bảo bảo mật đa tầng: JWT, RBAC, Row-Level Isolation theo phạm vi cửa hàng;
5. Triển khai toàn diện bằng Docker Compose với khả năng tự phục hồi.

Phạm vi đề tài

- **Dữ liệu:** Bộ dữ liệu Favorita Grocery Sales Forecasting (Kaggle) — 54 cửa hàng, 4.100 SKU, 33 ngành hàng, giai đoạn 2016–2017;
- **Mô hình:** LightGBM, CatBoost (Gradient Boosting); LLM Qwen 3.6-27B (Groq Cloud);
- **Hạ tầng:** Docker Compose, FastAPI, React, SQLite WAL mode;
- **Chức năng:** Dự báo, dashboard trực quan hóa, chatbot AI hỗ trợ ra quyết định nhập hàng;
- **Đánh giá ngoài:** Submission lên Kaggle Leaderboard (public + private score).

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về học máy (Machine Learning)

1.1.1 Khái niệm

Học máy là lĩnh vực của trí tuệ nhân tạo hướng tới xây dựng các thuật toán và mô hình có khả năng “học” từ dữ liệu huấn luyện để đưa ra dự đoán hoặc quyết định mà không cần được lập trình chi tiết cho từng tình huống cụ thể.

Về bản chất, học máy tìm cách xấp xỉ một hàm ánh xạ $f : \mathcal{X} \rightarrow \mathcal{Y}$ từ không gian đầu vào $\mathcal{X}$ sang không gian đầu ra $\mathcal{Y}$ dựa trên tập dữ liệu huấn luyện $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$. Mục tiêu là tìm hàm $\hat{f}$ sao cho sai số tổng quát (generalization error) trên dữ liệu mới là nhỏ nhất.

1.1.2 Phân loại các bài toán học máy

Học có giám sát (Supervised Learning)

Mô hình được huấn luyện trên tập dữ liệu có gắn nhãn, học ánh xạ từ đầu vào sang đầu ra bằng cách giảm thiểu sai số so với nhãn thực tế:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f(x_i))$$

trong đó $\mathcal{L}$ là hàm mất mát (loss function). Ứng dụng điển hình: phân loại, hồi quy, dự báo chuỗi thời gian.

Học không giám sát (Unsupervised Learning)

Mô hình tự tìm cấu trúc ẩn trong dữ liệu không nhãn. Các kỹ thuật phổ biến: phân cụm (K-Means, DBSCAN), giảm chiều (PCA, t-SNE), mô hình sinh (VAE, GAN).

Học tăng cường (Reinforcement Learning)

Tác nhân tương tác với môi trường, nhận phần thưởng/hình phạt và học chính sách hành động tối ưu tích lũy:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \mid \pi \right]$$

Ứng dụng: AlphaGo, điều khiển robot, xe tự lái, tối ưu hóa chuỗi cung ứng.

1.1.3 Bài toán hồi quy chuỗi thời gian

Trọng tâm của đề tài là **bài toán hồi quy chuỗi thời gian có giám sát** phục vụ dự báo doanh số bán hàng. Cho chuỗi lịch sử $\{y_1, y_2, \dots, y_t\}$, bài toán là dự đoán y_{t+h} với h là khoảng dự báo (horizon). Trong đề tài, $h = 16$ ngày.

1.2 Bài toán dự báo chuỗi thời gian (Time Series Forecasting)

1.2.1 Định nghĩa

Chuỗi thời gian là chuỗi các điểm dữ liệu xảy ra theo thứ tự liên tiếp trong một khoảng thời gian, phản ánh chuyển động của đại lượng quan sát (ví dụ doanh số bán hàng ngày):

$$\{y_t\}_{t=1}^T = \{y_1, y_2, \dots, y_T\}$$

1.2.2 Các thành phần của chuỗi thời gian

Một chuỗi thời gian thường được phân rã thành bốn thành phần:

- **Xu hướng (Trend):** xu hướng tổng thể tăng/giảm theo thời gian;
- **Mùa vụ (Seasonality):** dao động lặp lại theo chu kỳ cố định (theo tuần, tháng, quý; dịp lễ, mùa mưa nắng);
- **Chu kỳ (Cycle):** vận động trong khoảng dài hơn, không cố định chu kỳ như mùa vụ;
- **Yếu tố bất thường (Irregular/White Noise):** nhiễu còn lại sau khi trích các thành phần trên.

Mô hình phân rã cộng:

$$y_t = T_t + S_t + C_t + \epsilon_t$$

hoặc phân rã nhân:

$$y_t = T_t \times S_t \times C_t \times \epsilon_t$$

1.2.3 Các phương pháp dự báo truyền thống

ARIMA (AutoRegressive Integrated Moving Average)

Mô hình ARIMA(p, d, q) kết hợp:

- AR(p): Tự hồi quy — giá trị hiện tại phụ thuộc p giá trị quá khứ;
- I(d): Sai phân d lần để chuỗi dừng (stationary);
- MA(q): Trung bình trượt — giá trị hiện tại phụ thuộc q sai số quá khứ.

$$\phi(B)(1 - B)^d y_t = \theta(B)\epsilon_t$$

Phù hợp chuỗi đơn biến ổn định nhưng hạn chế khi cần biến ngoại sinh hoặc dự báo đồng thời hàng nghìn chuỗi (sản phẩm $\times$ cửa hàng).

SARIMA/SARIMAX

Mở rộng ARIMA có thành phần mùa vụ và biến ngoại sinh:

$$\phi(B)\Phi(B^s)(1 - B)^d(1 - B^s)^D y_t = \theta(B)\Theta(B^s)\epsilon_t$$

Exponential Smoothing (Holt–Winters)

Làm tròn mũ gán trọng số giảm dần cho quan sát cũ:

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha)\hat{y}_t$$

Vẫn giới hạn đơn biến, khó xử lý quan hệ phi tuyến giữa nhiều biến ngoại sinh.

1.2.4 Học máy trong dự báo chuỗi thời gian

Cách tiếp cận hiện đại chuyển bài toán dự báo thành bài toán học có giám sát bằng feature engineering:

Biến trễ (Lag Features)

Nắm bắt tự tương quan trong chuỗi:

$$x_t^{\text{lag}} = y_{t-l}, \quad l \in \{7, 14, 28\}$$

Độ trễ tối thiểu 7 ngày bảo đảm tính khả thi khi dự báo chu kỳ 16 ngày không rõ dữ liệu.

Thống kê trượt (Rolling Window Statistics)

Khử nhiễu cục bộ, nắm bắt xu hướng ngắn hạn:

$$x_t^{\text{roll}} = \frac{1}{W} \sum_{i=1}^W y_{t-i}, \quad W = 7$$

Đặc trưng lịch biểu – mùa vụ

Thứ trong tuần, tháng, cờ cuối tuần; đặc trưng trước/sau ngày lễ ($\pm 1, \pm 2$ ngày) và cờ mùa tựu trường đặc thù Ecuador (bang Costa: tháng 4–5; bang Sierra: tháng 8–9).

Đặc trưng ngoại sinh

Giá dầu thô quốc tế (đo lường sức mua vĩ mô), tỷ lệ sản phẩm được khuyến mãi, số giao dịch cửa hàng trễ 1 ngày.

1.3 Các mô hình học máy sử dụng trong dự báo

1.3.1 Mô hình cây tăng cường Gradient (GBDT)

Cây quyết định là phương pháp học có giám sát phi tham số cho phân loại và hồi quy, dự đoán bằng cách học các quy tắc quyết định từ đặc trưng dữ liệu.

Nguyên lý Gradient Boosting

LightGBM và CatBoost cùng dựa trên khung Gradient Boosting: tại vòng lặp thứ t , mô hình tìm cây f_t tối thiểu hóa hàm mục tiêu:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

trong đó $\Omega(f)$ là số hạng phạt độ phức tạp cây:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Khai triển Taylor bậc hai quanh dự báo hiện tại:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

với gradient bậc nhất và bậc hai:

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)2}}$$

Điểm đột phá của LightGBM

Phát triển lá (Leaf-wise Growth). LightGBM chọn nút lá có độ giảm mất mát lớn nhất để phân tách thay vì mở rộng đều theo tầng (level-wise), cho độ sâu hiệu dụng cao hơn với cùng số nút.

GOSS (Gradient-based One-Side Sampling). Giữ toàn bộ top $a \cdot n$ mẫu gradient lớn, lấy mẫu ngẫu nhiên $b \cdot n$ mẫu gradient nhỏ và bù trọng số $\frac{1-a}{b}$ để không lệch phân phối:

$$\tilde{V} = \frac{1}{n} \left[\sum_{g \in A_l} |g_g| + \frac{1-a}{b} \sum_{g \in A_b} |g_g| \right]$$

EFB (Exclusive Feature Bundling). Dùng tô màu đồ thị để gộp các đặc trưng thưa hiếm khi đồng thời khác 0, giảm độ phức tạp từ $O(n \times n_{\text{feat}})$ xuống $O(n \times n_{\text{bundle}})$.

Điểm đột phá của CatBoost

Ordered Target Statistics. Mã hóa biến phân loại theo thống kê đích có thứ tự để tránh rò rỉ mục tiêu (target leakage):

$$\widetilde{\text{TS}}_{\sigma}(c) = \frac{\sum_{k < i} [x_{k,c} = c] y_k + \alpha \bar{p}}{\sum_{k < i} [x_{k,c} = c] + \alpha}$$

với hoán vị ngẫu nhiên σ , giá trị tiên nghiệm $\bar{p}$ và trọng số làm trơn Laplace α .

Cây đối xứng (Oblivious Trees). Mọi nút cùng tầng dùng chung điều kiện phân nhánh, hoạt động như bảng tra cứu nhị phân — suy luận nhanh và chống quá khớp.

1.3.2 Thuật toán phân cụm (K-Means)

K-Means là phương pháp học không giám sát phân chia n quan sát thành K cụm $\{S_1, \dots, S_K\}$ bằng cách tối thiểu hóa tổng bình phương khoảng cách nội cụm (WCSS):

$$\arg \min_S \sum_{j=1}^K \sum_{x \in S_j} \|x - \mu_j\|^2$$

với μ_j là tâm cụm j .

Lưu ý áp dụng trong đề tài: Hệ thống không tự chạy K-Means trên dữ liệu bán hàng mà tận dụng sẵn nhãn phân cụm (cluster) đi kèm `stores.csv` của Favorita (54 cửa hàng thuộc 17 cụm), sau đó áp dụng kỹ thuật mã hóa đích có làm trơn (Smoothed Target Encoding) để chuyển nhãn cụm thành đặc trưng thống kê cho mô hình.

1.3.3 Mô hình hóa đa cấp: đánh đổi độ lệch – phương sai

Sự phân chia giữa 33 Local Models và Global Fallback Model xuất phát từ nguyên lý bias–variance tradeoff:

- **Global Model (mô hình gộp toàn chuỗi):** phương sai thấp nhờ học trên toàn bộ dữ liệu lớn, nhưng độ chệch cao với các ngành hàng cá biệt do “bình quân hóa” (regression to the mean);

- **Local Models (từng ngành hàng):** độ chệch rất thấp do bám sát phân phối riêng từng ngành hàng, nhưng phương sai cao nếu dữ liệu ngành đó quá ít;
- **Chiến lược Smart Routing & Ensemble:** dùng Local Model làm tuyến chính và Global Ensemble (LightGBM + CatBoost) làm tuyến dự phòng, tối thiểu hóa sai số toàn cục.

1.3.4 Kiến trúc Transformer và LLM Agent

Cơ chế Self-Attention

Kiến trúc Transformer là nền tảng của các mô hình ngôn ngữ lớn (LLM). Cơ chế tự chú ý (Self-Attention) tính biểu diễn mỗi token bằng tổng hợp có trọng số từ toàn bộ ngữ cảnh:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Multi-Head Attention

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

LLM Agent với Function Calling

Trong đề tài, mô hình qwen/qwen3.6-27b (27 tỷ tham số) được khai thác qua API của Groq Cloud — bộ vi xử lý LPU (Language Processing Unit) chuyên dụng cho suy luận ngôn ngữ độ trễ thấp.

Năng lực tư vấn được khuếch đại bằng kỹ thuật **Function Calling**: thay vì để LLM suy đoán số liệu, hệ thống định nghĩa bộ hàm công cụ mà mô hình chủ động triệu gọi; kết quả từ cơ sở dữ liệu và các hàm toán chuỗi cung ứng được đưa trở lại ngữ cảnh hội thoại để mô hình tổng hợp câu trả lời cuối, hạn chế hiện tượng ảo giác (hallucination).

1.3.5 Chỉ số đánh giá mô hình dự báo

RMSLE (Root Mean Squared Logarithmic Error)

Chỉ số mục tiêu chính:

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\ln(1 + \hat{y}_i) - \ln(1 + y_i))^2}$$

Đặc tính:

- Đo sai số tương đối theo tỉ lệ

- Phạt nặng hơn dự báo thiếu so với dự báo thừa (phù hợp tránh đứt hàng)
- Khử ảnh hưởng các đỉnh doanh số bất thường.

NWRMSLE (Normalized Weighted RMSLE)

Chỉ số chính thức của cuộc thi Kaggle Favorita — mở rộng RMSLE với trọng số w_i cho từng nhóm sản phẩm (hàng dễ hỏng *perishable* có trọng số 1.25):

$$\text{NWRMSLE} = \sqrt{\frac{\sum_{i=1}^n w_i (\ln(1 + \hat{y}_i) - \ln(1 + y_i))^2}{\sum_{i=1}^n w_i}}$$

Chỉ số này được dùng để chấm điểm trên Kaggle Leaderboard (public + private).

MAE (Mean Absolute Error)

Sai số tuyệt đối trung bình theo đơn vị sản phẩm - con số trực quan cho nhân viên thủ kho:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

RMSE (Root Mean Squared Error)

Nhạy với sai số lớn, giám sát ngành hàng tốc độ bán cao (GROCERY I, BEVERAGES):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

MAPE (Mean Absolute Percentage Error)

Chỉ số đo sai số dự báo theo phần trăm so với giá trị thực tế:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%$$

Hạn chế: MAPE không xác định khi tồn tại $y_i = 0$ — rất phổ biến trong dữ liệu bán lẻ theo ngày — và phạt bất đối xứng, nặng hơn khi y_i nhỏ, khiến chỉ số bị lệch bởi các quan sát có giá trị thực tế gần 0.

WAPE (Weighted Absolute Percentage Error)

Từ hạn chế của MAPE nêu trên, WAPE được đề xuất như một giải pháp thay thế, khắc phục lỗi chia cho $y_i = 0$ bằng cách chuẩn hoá tổng sai số tuyệt đối theo tổng giá trị thực tế thay vì theo từng quan sát riêng lẻ:

$$\text{WAPE} = \frac{\sum_{i=1}^n |\hat{y}_i - y_i|}{\sum_{i=1}^n y_i}$$

Đây là chỉ số được ưu tiên trình bày trong báo cáo cho người dùng cuối (chủ cửa hàng) trên nền tảng, vì thể hiện sai số dưới dạng phần trăm dễ hiểu so với tổng doanh số thực tế, thay vì đơn vị log như RMSLE.

WMAPE (Weighted Mean Absolute Percentage Error)

Phiên bản có trọng số của WAPE, kế thừa tinh thần weighting của NWRMSLE: hàng dễ hỏng (*perishable*) được nhân trọng số $w_i = 1,5$ vì rủi ro đứt hàng của nhóm này cao hơn, khiến sai số ở nhóm “xấu” bị phạt nặng hơn trong tổng thể:

$$\text{WMAPE} = \frac{\sum_{i=1}^n w_i |\hat{y}_i - y_i|}{\sum_{i=1}^n w_i |y_i|}, \quad w_i = \begin{cases} 1.5 & \text{nếu sản phẩm dễ hỏng} \\ 1.0 & \text{ngược lại} \end{cases}$$

Khi $w_i = 1$ với mọi quan sát, WMAPE trùng khít với WAPE — nhờ đó hai chỉ số luôn có thể so sánh trực tiếp: WMAPE thấp hơn WAPE cho thấy mô hình đạt sai số tương đối thấp hơn chính ở nhóm hàng có trọng số cao (nhóm quan trọng nhất về độ tươi hàng hóa).

Forecast Accuracy (Accuracy classification score)

Chỉ số mang ý nghĩa “tỷ lệ dự đoán đúng” của bài toán phân loại, quy đổi từ WAPE:

$$\text{Forecast Accuracy} = \max(0, 1 - \text{WAPE}) = \max\left(0, 1 - \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n y_i}\right)$$

Đây là chỉ số của bài toán phân loại, không phù hợp áp dụng trực tiếp cho bài toán hồi quy dự báo doanh số liên tục như trong đề tài, do đó không được dùng làm chỉ số đánh giá chính.

R^2 (Coefficient of determination) regression score function.

Thông thường, giá trị R^2 nằm trong khoảng $[0, 1]$ đối với tập huấn luyện, nhưng trên tập kiểm thử (test set) giá trị này có thể nhận giá trị âm $((-\infty, 1])$.

Hệ số xác định tổng quát được định nghĩa theo công thức:

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Trong hệ thống, R^2 được dùng như chỉ số tham chiếu bổ sung để đánh giá mức độ mô hình giải thích được biến động doanh số tổng thể, đặc biệt hữu ích khi so sánh giữa các fold walk-forward có quy mô dữ liệu huấn luyện khác nhau.

1.4 Nền tảng web và kiến trúc hệ thống hỗ trợ ra quyết định

1.4.1 Hệ thống hỗ trợ ra quyết định (DSS)

Hệ thống hỗ trợ ra quyết định giúp nhà quản lý tổng hợp, phân tích dữ liệu và trực quan hóa thông tin:

- Dashboard xu hướng doanh số theo thời gian;
- Cảnh báo bất thường tồn kho;
- Gợi ý số lượng đặt hàng/nhập kho;
- Phân tích kịch bản (what-if analysis).

1.4.2 Kiến trúc ứng dụng web hiện đại

- **Tầng giao diện (Frontend):** React 18 + Vite, Tailwind CSS v4, Radix UI / Shadcn UI;
- **Tầng ứng dụng (Backend Gateway):** FastAPI xử lý logic nghiệp vụ, API RESTful, bảo mật, điều phối LLM Agent;
- **Tầng suy luận học máy (ML Service):** Microservice FastAPI độc lập phục vụ 33 Local models và Global Ensemble;
- **Tầng dữ liệu (Data layer):** SQLite WAL mode lưu trữ giao dịch lịch sử, dữ liệu dự báo, mức tồn kho và các bảng cache tổng hợp.

1.5 Tổng quan tập dữ liệu Favorita Grocery Sales Forecasting

Các cửa hàng tạp hóa truyền thống luôn phải cân nhắc kỹ lưỡng giữa việc mua hàng và dự báo doanh số. Nếu dự đoán cao hơn một chút, các cửa hàng sẽ bị tồn kho hàng hóa

dễ hư hỏng. Nếu dự đoán thấp hơn một chút, các mặt hàng phổ biến sẽ nhanh chóng bán hết, dẫn đến mất doanh thu và khách hàng tức giận.

Vấn đề trở nên phức tạp hơn khi các nhà bán lẻ mở thêm địa điểm mới với những nhu cầu riêng biệt, sản phẩm mới, thị hiếu theo mùa luôn thay đổi và tiếp thị sản phẩm khó lường. Corporación Favorita, một nhà bán lẻ tạp hóa lớn có trụ sở tại Ecuador, hiểu rõ điều này hơn ai hết. Họ điều hành hàng trăm siêu thị, với hơn 200.000 sản phẩm khác nhau trên kệ.

Công ty Corporación Favorita đã thách thức cộng đồng Kaggle xây dựng một mô hình dự báo doanh số sản phẩm chính xác hơn. Hiện tại, họ đang dựa vào các phương pháp dự báo chủ quan với rất ít dữ liệu hỗ trợ và rất ít tự động hóa để thực hiện kế hoạch. Họ rất hào hứng muốn xem máy học có thể giúp họ đảm bảo làm hài lòng khách hàng tốt hơn bằng cách cung cấp đủ lượng sản phẩm phù hợp vào đúng thời điểm như thế nào.

Tập dữ liệu do Corporación Favorita (Ecuador) công bố trên Kaggle gồm:

Tệp	Kích thước	Nội dung
train.csv	≈4.77 GB	Doanh số theo ngày × cửa hàng × mặt hàng
stores.csv	Nhỏ	54 cửa hàng, 22 thành phố, 17 cụm, 5 loại hình
items.csv	Nhỏ	4.100 SKU thuộc 33 ngành hàng, cờ hàng để hỏng
transactions.csv	Trung bình	Số giao dịch theo ngày mỗi cửa hàng
oil.csv	Nhỏ	Giá dầu hằng ngày (43 giá trị thiếu)
holidays_events.csv	Nhỏ	Ngày lễ, sự kiện đặc biệt

Bảng 1.1: Tổng quan các tệp dữ liệu trong bộ Favorita.

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu hệ thống

2.1.1 Khảo sát bài toán thực tế

Qua khảo sát vận hành, doanh nghiệp bán lẻ vừa và nhỏ đối diện bốn nhu cầu cấp thiết:

1. **Khủng hoảng quản trị tồn kho:** Mắc kẹt giữa cháy hàng (mất doanh thu, giảm lòng trung thành) và đọng vốn (chi phí lưu kho, hao hụt hàng để hỏng). *Nhu cầu:* Cơ chế cảnh báo tồn kho chủ động, tự tính ROP và Safety Stock theo tốc độ tiêu thụ thực tế.
2. **Hạn chế dự báo truyền thống:** Dự báo “cảm tính” hay trung bình Excel bỏ qua yếu tố ngoại sinh và mùa vụ. *Nhu cầu:* Mô hình LightGBM/CatBoost dự báo ngắn hạn 16 ngày ở cấp ngành hàng và từng SKU.
3. **Thiếu nhân lực phân tích dữ liệu:** SME không đủ ngân sách thuê đội ngũ data scientist. *Nhu cầu:* Hệ thống SaaS “cắm là chạy”, mô hình ML ẩn sau API.
4. **Khoảng cách giữa con số và hành động:** Phần mềm dự báo chỉ trả số liệu. *Nhu cầu:* Trợ lý AI hiểu ngôn ngữ tự nhiên, tự gọi thuật toán ML và toán chuỗi cung ứng để đưa gợi ý hành động cụ thể.

2.1.2 Xác định đối tượng người dùng (User Roles & Personas)

Hệ thống thiết kế theo mô hình B2B SaaS, phân ba cấp người dùng theo RBAC:

Vai trò	Chức năng chính	Phạm vi truy cập
Quản trị viên hệ thống (SaaS Provider)	Vận hành hạ tầng microservices, giám sát sức khỏe dịch vụ, tái huấn luyện mô hình định kỳ	Toàn bộ hệ thống nhưng không can thiệp dữ liệu kinh doanh hằng ngày
Quản lý doanh nghiệp (Tenant Manager)	Dashboard tổng quan toàn bộ cửa hàng, so sánh cụm cửa hàng, phân tích kịch bản vĩ mô	Toàn bộ dữ liệu của doanh nghiệp mình
Quản lý cửa hàng (Store Manager)	Dự báo chi tiết cửa hàng mình, cảnh báo cháy hàng, tính ROP và số lượng nhập từng SKU	Row-Level Isolation theo store_nbr gắn tài khoản

Bảng 2.1: Phân vai người dùng và phạm vi quyền hạn.

2.1.3 Yêu cầu kỹ thuật và ràng buộc vận hành

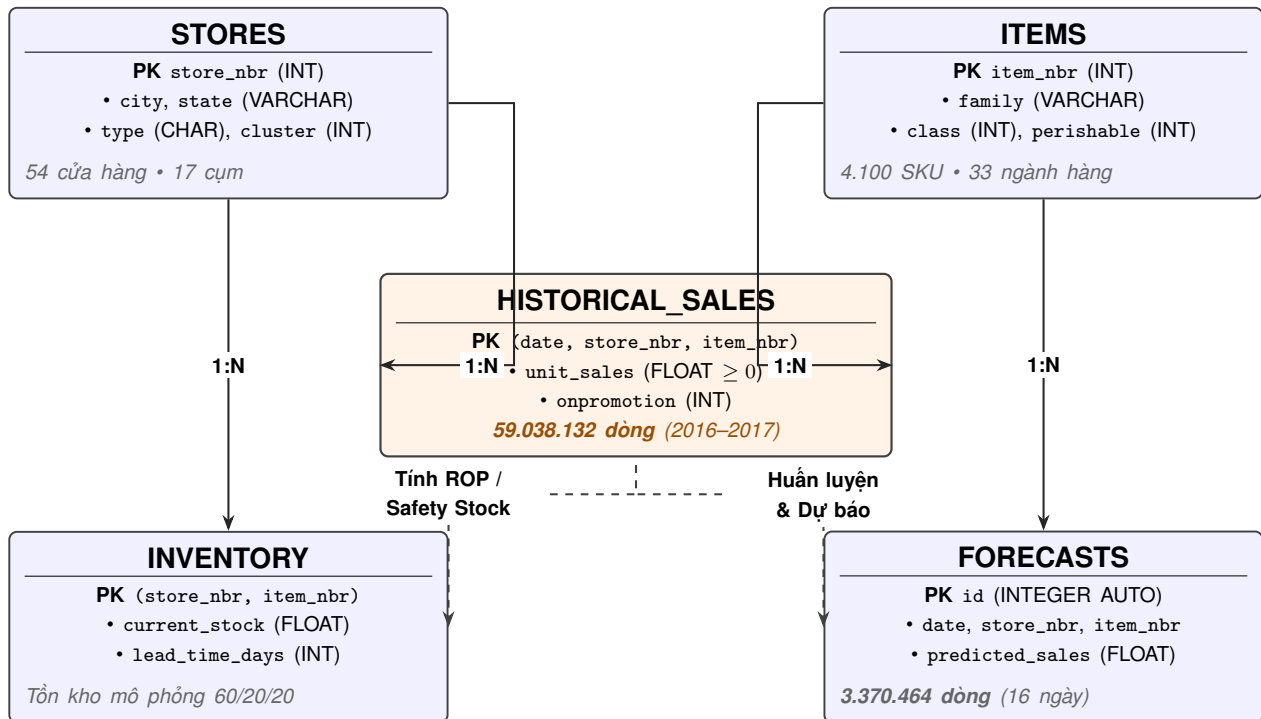
- **Hiệu năng:** Endpoint truy vấn CSDL phản hồi dưới 1 giây (nhờ bảng tổng hợp cache); agent LLM phản hồi dưới 3 giây trên Groq LPU, tổng thời gian chat dưới 5 giây; suy luận real-time dưới 2 giây nhờ preload trọng số lúc khởi động service.
- **Khả năng mở rộng:** Kiến trúc microservices cho phép scale ngang từng dịch vụ; pipeline offline xử lý file CSV hàng GB bằng chia khối (chunking) tránh tràn RAM.
- **Bảo mật:** JWT cho mọi request frontend; RBAC + Row-Level Isolation; SQLite mode read-only cho lớp công cụ LLM; API key quản lý bằng biến môi trường, không hard-code.
- **Chịu lỗi:** Fallback Local Model → Global Ensemble ở tầng ML; graceful degradation ở tầng agent (HTTP 502 kèm thông điệp tiếng Việt); Docker restart: always tự phục hồi.
- **Bảo trì:** Module hóa (data_loader, preprocessor, agent, tools); Swagger/OpenAPI tự động tại <http://localhost:8000/docs>; tách biệt ml_training (offline) và ml_service (online) cho phép retrain không dừng server.

2.2 Kiến trúc tổng thể hệ thống

Hệ thống tổ chức theo mô hình microservices đóng gói bằng Docker Compose trên mạng bridge riêng retail_network: dịch vụ suy luận học máy ml_service (cổng 8001), cổng API backend (cổng 8000) và giao diện web frontend (cổng 8501 ánh xạ cổng 80 nginx

trong container).

Các dịch vụ phụ thuộc theo chuỗi có điều kiện sức khỏe (depends_on với condition: service_healthy) bảo đảm thứ tự khởi động: mô hình nạp xong thì backend mới mở cổng, backend khỏe thì frontend mới phục vụ.



Hình 2.1: Sơ đồ ERD rút gọn của retail.db (kèm quy mô dòng đo được).

Trọng số mô hình không đóng gói vào image mà mount vào container chế độ chỉ đọc (:ro), cho phép nâng cấp mô hình không cần build lại image. Hai dịch vụ Python cấu hình healthcheck định kỳ và chính sách restart: always.

2.3 Thiết kế pipeline dữ liệu và huấn luyện mô hình

Pipeline huấn luyện tách khỏi serving theo nguyên tắc offline training / online inference, gồm sáu bước nối tiếp trong ml_training/src:

Phạm vi	Công nghệ	Vai trò chính
Backend API	Python 3.11, FastAPI, Uvicorn	Cổng RESTful, xác thực, điều phối nghiệp vụ
ML Service	FastAPI, LightGBM 4.x, CatBoost, scikit-learn, joblib	Serving 33 local model và global ensemble
LLM Agent	Groq API — qwen/qwen3.6-27b (LPU)	Trợ lý tư vấn chuỗi cung ứng qua Function Calling
Frontend	React 18, Vite 6, TypeScript, Tailwind CSS v4, Recharts, Shadcn	Dashboard, biểu đồ, giao diện chat
Web server	nginx (alpine)	Tệp tĩnh, reverse proxy /api, gzip
CSDL	SQLite (WAL), chỉ mục tổ hợp	59 triệu dòng lịch sử, dự báo, tồn kho
Bảo mật	PyJWT (HS256), PBKDF2-HMAC-SHA256 200k vòng	Token phiên, băm mật khẩu, API key ERP
Triển khai	Docker Compose, healthcheck	Đóng gói đồng nhất, tự phục hồi

Bảng 2.2: Công nghệ và công cụ sử dụng trong hệ thống.

1. `data_loader.py`: Đọc `train.csv` gốc (≈ 4.77 GB cấp item), lọc từ 01/01/2016, hợp nhất `items/stores/oil/holidays/transactions`, tổng hợp về cấp độ (cửa hàng $\times$ ngành hàng $\times$ ngày); cache Parquet 874.052 dòng $\times$ 15 cột;
2. `preprocessor.py`: Sinh đặc trưng (lag, rolling, lịch, lễ, đầu) dùng chung huấn luyện và suy luận — feature store nhất quán;
3. `cluster_features.py`: `ClusterFeatureEngineer` — mã hóa đích có làm trơn cho đặc trưng cụm, xuất `cluster_engineer.pkl`;
4. `train.py`: Global ensemble LightGBM + CatBoost kiểm định walk-forward 3-fold, tự tối ưu trọng số hòa trộn, xuất artifact và `ensemble_meta.json`;
5. `train_local_models.py`: 33 mô hình LightGBM cấp ngành hàng + `local_models_metadata.json`;
6. `predict_local.py` + `load_forecasts_inventory.py` + `init_database.py`: Dự báo độ quy 16 ngày toàn chuỗi, phân rã xuống SKU, nạp `retail.db` kèm tồn kho mô phỏng.

Artifact mô hình đặt tại `ml_service/models` và mount chỉ đọc:

- `lgbm_model.pkl` (≈ 14 MB)
- `catboost_model.cbm` (≈ 19 MB)
- `local_lgbm_models.pkl` (≈ 6.8 MB gồm 33 mô hình kèm preprocessor từng ngành)
- `preprocessor.pkl`, `cluster_engineer.pkl`
- `ensemble_meta.json`, `metrics.json`, `local_models_metadata.json`

2.4 Thiết kế trợ lý AI thông minh (LLM Agent)

Agent hai pha: Backend gửi hội thoại tới qwen/qwen3.6-27b (Groq Cloud) cùng System Prompt vai trò “Chuyên gia quản trị chuỗi cung ứng bán lẻ” và danh sách công cụ; mô hình chọn hàm cần gọi (vòng 1), backend thực thi trên CSDL rồi trả JSON về cho mô hình tổng hợp câu trả lời (vòng 2).

Tham số sinh: temperature = 0.6; max_tokens = 2048. Lỗi Groq ánh xạ thành HTTP 502 kèm detail tiếng Việt.

2.4.1 Bộ công cụ và toán học chuỗi cung ứng

Chín công cụ Function Calling bao phủ truy vấn dữ liệu và tính toán nghiệp vụ; mọi câu SQL viết tĩnh trong `tools.py` (LLM không tự sinh SQL).

Công thức Điểm đặt hàng lại (ROP)

$$ROP = \bar{d} \cdot LT + z \sigma \sqrt{LT}$$

Trong đó:

- $\bar{d}$: Nhu cầu trung bình/ngày;
- LT : Lead time (thời gian giao hàng);
- $z = 1.65$ (mức phục vụ 95%);
- $\sigma \approx 0.2 \bar{d}$: Độ lệch chuẩn nhu cầu.

Số lượng đề xuất nhập

$$Q_{\text{suggested}} = \max(0, \hat{F}_{16d} - \text{Stock}_{\text{hiện tại}})$$

Rủi ro cháy hàng

Tồn kho < (dự báo ngày × lead time) ⇒ cảnh báo gấp; thiệt hại cơ hội theo giá giả định 5 USD/đơn vị, vượt 100 USD xếp mức nguy hiểm.

Hàng chậm luân chuyển

$\hat{F}_{16d} < \text{Stock}/2$ và $\text{Stock} > 20 \Rightarrow$ đề xuất giảm giá 15%/30%.

Gợi ý bán chéo

Top 3 sản phẩm cùng ngành hàng doanh số cao nhất; so sánh cụm cửa hàng: chênh lệch % doanh số giữa cụm người dùng và trung bình toàn chuỗi.

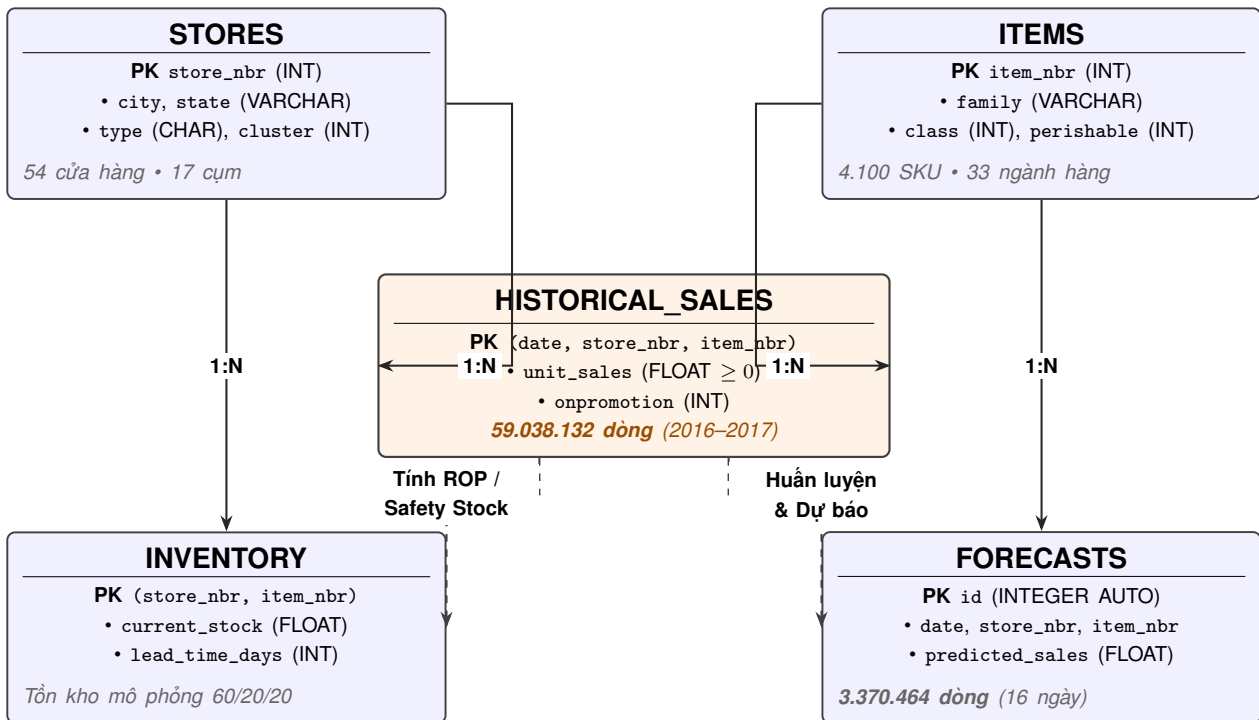
2.4.2 Ràng buộc an toàn của tác tử

- Kết nối dữ liệu agent dùng SQLite mode=ro (chỉ đọc) — về nguyên tắc không phát sinh lệnh ghi/xóa hỏng dữ liệu;
- validate_tool_access kiểm tra quyền cửa hàng trước mỗi lần gọi công cụ; ngoài phạm vi bị từ chối ngay, thông báo đưa trở lại ngữ cảnh cho mô hình giải thích;
- Ngoại lệ công cụ đóng gói JSON trả về mô hình, không đứt gãy hội thoại.

2.5 Thiết kế cơ sở dữ liệu

2.5.1 Mô hình thực thể – quan hệ (ERD)

CSDL SQLite thiết kế chuẩn hóa gồm các thực thể chính: STORES, ITEMS, HISTORICAL_SALES, INVENTORY, FORECASTS cùng hai bảng tổng hợp cache (AGG_ITEM_STORE_SALES, AGG_FORECAST_DATE_FAMILY) và SKU_STATS, MODEL_META.



Hình 2.2: Sơ đồ ERD rút gọn của retail.db (kèm quy mô dòng đo được).

2.5.2 Mô tả các thực thể và thuộc tính

- **STORES**: PK store_nbr; thuộc tính city, state, type, cluster (54 cửa hàng, 22 thành phố, 17 cụm, 5 loại hình).
- **ITEMS**: PK item_nbr; family, class, perishable (4.100 SKU, 33 ngành hàng).
- **HISTORICAL_SALES**: Khóa chính tổ hợp (date, store_nbr, item_nbr); unit_sales

- (âm đã chuyển 0), onpromotion. Nạp bằng streaming chunking kết hợp executemany; quy mô 59.038.132 dòng (01/01/2016 — 15/08/2017).
- **INVENTORY:** PK (store_nbr, item_nbr); current_stock, lead_time_days, last_updated. Mô phỏng theo tốc độ tiêu thụ, phân bổ 60%/20%/20% bình thường/sắp hết/động vốn, lead time ngẫu nhiên 1–7 ngày (trung bình ≈ 1 ngày) để demo agent.
 - **FORECASTS:** PK id; date, store_nbr, item_nbr, predicted_sales. 16 ngày dự báo 16/08—31/08/2017, tổng 3.370.464 dòng.
 - **AGG_ITEM_STORE_SALES:** Bảng cache WITHOUT ROWID, PK (store_nbr, item_nbr), 172.130 dòng — giúp API phân tích phản hồi tức thì không quét 59 triệu dòng lịch sử.
 - **AGG_FORECAST_DATE_FAMILY:** Cache dự báo theo ngày $\times$ ngành hàng (28.512 dòng) phục vụ biểu đồ xu hướng.
 - **SKU_STATS / MODEL_META:** Thống kê từng SKU (doanh số 2016, avg_daily_45d, phân lớp ABC, family RMSLE) và siêu dữ liệu mô hình (global_rmsle, sku_validation).

2.5.3 Tối ưu hóa cơ sở dữ liệu

- **Chỉ mục tổ hợp:** idx_forecasts_store_date (store_nbr, date), idx_hist_store_date, idx_agg_item, idx_inventory_store;
- **Read-only URI:** Lớp tools/dashboard/products kết nối file: ...?mode=ro chống ghi/xóa vô tình;
- **WITHOUT ROWID** cho bảng ánh xạ khóa chính tổ hợp — giảm dung lượng, tăng tốc tra cứu;
- **PRAGMA journal_mode=WAL:** Đọc song song không block ghi.

2.6 Thiết kế giao diện người dùng (UI/UX)

2.6.1 Sơ đồ luồng màn hình

Luồng sử dụng chính: **Đăng nhập** (JWT lưu localStorage) → **Chọn chi nhánh** (BranchBar liệt kê cửa hàng theo quyền từ /api/stores) → bốn màn hình chức năng:

- *Dự báo doanh số* (SalesForecast)
- *Phân tích sản phẩm* (ProductAnalysis)
- *Quản lý sản phẩm* (ProductManagement)
- *Trợ lý AI* (AIChatbot)

Toàn bộ nhãn giao diện tiếng Việt.

2.6.2 Thiết kế dashboard trực quan hóa dự báo

Màn hình Dự báo doanh số

- Thẻ KPI tổng quan (/api/kpi);
- Siêu dữ liệu kỳ dự báo (/api/forecast-meta);
- Biểu đồ đường doanh số dự báo 16 ngày theo cửa hàng (Recharts LineChart, /api/predictions);
- Bảng Top 6 sản phẩm bán chạy (/api/top-products?limit=6).

Màn hình Phân tích sản phẩm

- Pie chart thị phần ngành hàng (/api/family-mix);
- Line chart nhiều chuỗi xu hướng dự báo theo ngành (/api/family-trend);
- Bảng danh mục 4.100 SKU với tìm kiếm, lọc ngành hàng, phân trang server-side (/api/products; giao diện gửi page_size = 15 dòng/trang, API cho phép tối đa 100).

2.6.3 Thiết kế chức năng hỗ trợ ra quyết định

- **Cảnh báo tồn kho:** Đếm SKU tồn thấp ($0 < \text{stock} < 30$) hiển thị trên danh mục; trạng thái restock từng sản phẩm;
- **Gợi ý nhập hàng:** Nút đặt hàng lại gọi POST /api/inventory/restock;
- **Trợ lý AI:** Hộp chat với câu hỏi nhanh dựng sẵn (phân tích cửa hàng, rủi ro cháy hàng, hàng tồn chậm, gợi ý nhập, bán chéo); gửi tối đa 16 lượt hội thoại gần nhất làm ngữ cảnh; câu trả lời dạng markdown có dẫn số liệu cụ thể (mã hàng, tồn kho, dự báo, số ngày an toàn);
- **Phân tích ABC:** Gắn nhãn lớp A/B/C theo đóng góp doanh số trong sku_stats hỗ trợ ưu tiên quản trị.

2.7 Thiết kế bảo mật và phân quyền

2.7.1 Cơ chế xác thực

Đăng nhập bằng username/password; mật khẩu băm **PBKDF2-HMAC-SHA256 200.000 vòng, salt 16 byte**. Phiên đăng nhập cấp **JWT HS256** hết hạn 480 phút, secret qua biến môi trường.

Hỗ trợ song song xác thực API key (X-API-Key so khớp ERP_API_KEY) cho tích hợp ERP máy-máy, định danh admin hệ thống erp-integration.

Người dùng seed:

- admin/admin123 (tổng bộ 54 cửa hàng)
- manager1/manager123 (cửa hàng 1–10)
- manager2/manager123 (cửa hàng 11–20)

2.7.2 Phân quyền theo vai trò (RBAC) và theo tenant

Mọi endpoint (trừ login) yêu cầu token hợp lệ; quyền truy vấn dữ liệu gắn danh sách `allowed_stores` của tài khoản: admin truy cập toàn chuỗi, quản lý doanh nghiệp giới hạn theo tenant, quản lý cửa hàng giới hạn theo cửa hàng.

2.7.3 Đảm bảo cách ly dữ liệu giữa các tenant

Row-Level Isolation thực hiện ba lớp:

1. Dependency `ensure_store_access/store_filter/_scope_sql` chèn điều kiện phạm vi vào SQL và trả 403 kèm thông báo tiếng Việt khi vượt quyền;
2. `validate_tool_access` kiểm tra trước mỗi lần gọi công cụ của LLM (kể cả giải quyết `cluster` → cửa hàng qua kết nối `read-only`);
3. Ẩn danh sách cửa hàng được phép (`_allowed_stores`) inject vào hàm `get_sales_summary` khiến LLM không thể “nhìn thấy” dữ liệu ngoài phạm vi.

CHƯƠNG 3

HUẤN LUYỆN MÔ HÌNH VÀ TÍCH HỢP HỆ THỐNG

3.1 Môi trường và công cụ huấn luyện

Pipeline offline chạy trên Python với:

- **Xử lý dữ liệu:** pandas 2.x, NumPy;
- **Mô hình:** scikit-learn, LightGBM 4.x, CatBoost 1.2.x (huấn luyện CPU, loss RMSE);
- **Đóng gói:** joblib;
- **Tối ưu:** scipy (minimize_scalar cho blend weight).

Huấn luyện và serving tách môi trường độc lập: artifact build một lần rồi mount chỉ đọc vào container suy luận.

3.2 Nạp và làm sạch dữ liệu

Bộ dữ liệu gốc của cuộc thi Corporación Favorita có dung lượng khổng lồ (file `train.csv` nặng khoảng 5GB với hơn 125 triệu dòng giao dịch). Việc nạp và làm sạch dữ liệu này bằng phương pháp truyền thống (đọc toàn bộ vào RAM) sẽ dẫn đến lỗi tràn bộ nhớ (Out of Memory - OOM). Do đó, hệ thống đề xuất và triển khai một quy trình xử lý dữ liệu hiệu năng cao, bao gồm 4 giai đoạn chính:

3.2.1 Tối ưu hóa việc nạp dữ liệu thô (Streaming Ingestion)

Hệ thống áp dụng kỹ thuật **Streaming Chunking** kết hợp tối ưu kiểu dữ liệu (Dtype Optimization), tổ chức thành hai đường nạp riêng biệt:

- **Nạp CSDL theo khối (`init_database.py`):** Khi dựng `retail.db`, `train.csv` được đọc bằng `pd.read_csv(chunksize=2_000_000)` — mỗi phần (chunk) chứa 2.000.000 dòng; từng chunk được làm sạch rồi ghi ngay vào bảng `historical_sales` bằng `executemany()`, chỉ cộng dồn bộ đếm số dòng thay vì lưu trữ toàn bộ DataFrame. Cách này giữ cho dung lượng RAM sử dụng ở mức thấp ổn định (dưới 500MB) trong suốt quá trình chạy.
- **Đọc đầy đủ cho pipeline đặc trưng (`data_loader.py`):** Khâu sinh đặc trưng huấn

luyện đọc `train.csv` trọn vẹn một lần với dtype thu nhỏ, hợp nhất các bảng chiều, tổng hợp về cấp (cửa hàng $\times$ ngành hàng $\times$ ngày) rồi cache ra Parquet (`train_merged.parquet`) để các lần chạy sau không phải xử lý lại CSV gốc.

- **Ép kiểu dữ liệu (Dtype Casting):** Các cột số được ép từ `int64/float64` mặc định xuống `int16` (`store_nbr` khi nạp CSDL; `int8` ở pipeline đặc trưng), `int32` (`item_nbr`) và `float32` (`unit_sales`). Kỹ thuật này giúp giảm mức tiêu thụ RAM xuống khoảng 60%.
- **Phân luồng dữ liệu lịch sử:** Đối với dữ liệu quá khứ, cả hai đường nạp đều tự động lọc (filter) chỉ lấy các giao dịch từ ngày 01/01/2016 trở đi để giảm khối lượng tính toán, phù hợp với phạm vi dự báo ngắn hạn (16 ngày) của mô hình.

3.2.2 Làm sạch và Xử lý giá trị bất thường (Data Cleaning)

Quá trình làm sạch diễn ra trực tiếp trên từng chunk dữ liệu trước khi ghi vào CSDL, bao gồm các bước:

- **Xử lý ngày tháng (Date Parsing):** Thay vì để Pandas tự đoán định dạng ngày (dễ gây lỗi `ValueError` nếu có dòng dữ liệu hỏng), hệ thống dùng `pd.to_datetime(format="%Y-%m-%d", errors='coerce')`. Các dòng bị hỏng định dạng sẽ tự động biến thành giá trị rỗng (`NaT`) và bị loại bỏ, đảm bảo luồng dữ liệu không bị đứt quãng.
- **Xử lý doanh số âm (Negative Sales):** Trong thực tế bán lẻ, cột `unit_sales` có chứa giá trị âm (biểu thị cho việc khách hàng hoàn trả hàng). Để không làm sai lệch các phép tính trung bình trượt (Rolling Mean) và dự báo nhu cầu, hệ thống dùng hàm `.clip(lower=0)` để chuyển toàn bộ giá trị âm thành 0.
- **Điền giá trị thiếu (Missing Values Imputation):**
 - *Giá dầu (Oil Price):* Dữ liệu giá dầu không có vào các ngày cuối tuần. Hệ thống tạo một khung ngày liên tục và dùng phương pháp Forward Fill (`ffill()`) và Backward Fill (`bfill()`) để lấy giá dầu của thứ 6 áp dụng cho thứ 7, Chủ nhật.
 - *Khuyến mãi (Onpromotion):* Các ô trống được điền giá trị `False`, đảm bảo tính nhất quán của biến nhị phân.
- **Xử lý trùng lặp Ngày lễ (Holiday Deduplication):** Tập dữ liệu ngày lễ có thể có nhiều sự kiện cùng ngày tại các khu vực khác nhau (Quốc gia, Vùng, Địa phương). Nếu join trực tiếp sẽ làm số dòng dữ liệu nhân lên. Hệ thống xử lý bằng cách lọc `transferred == False` và loại trùng lặp theo ngày (ngày lễ Quốc gia dedup theo `date`; ngày Regional/Local dedup theo cặp (`date`, `locale_name`)) để đảm bảo mỗi ngày, mỗi khu vực chỉ tương ứng với 1 bản ghi ngày lễ duy nhất.

3.2.3 Mô phỏng dữ liệu tồn kho (Inventory Simulation)

Tập dữ liệu Kaggle Favorita không cung cấp thông tin tồn kho hiện tại và thời gian giao hàng (Lead Time) – những biến số bắt buộc phải có để tính toán Điểm đặt hàng lại (ROP) và Tồn kho an toàn (Safety Stock). Để giải quyết điểm yếu này, hệ thống xây dựng một module mô phỏng tồn kho dựa trên **Tốc độ tiêu thụ (Sales Velocity)**:

- Tính doanh số trung bình ngày (avg_daily_sales) của từng mặt hàng tại từng cửa hàng từ dữ liệu dự báo.
- Giả lập thời gian giao hàng (lead_time_days) ngẫu nhiên từ 1 đến 7 ngày.
- Phân bổ trạng thái tồn kho theo tỷ lệ thực tế: 60% mặt hàng ở mức bình thường, 20% ở trạng thái sắp hết hàng (Understock - dự trữ dưới 2 ngày), và 20% ở trạng thái đọng vốn (Overstock - dự trữ trên 30 ngày). Kỹ thuật này tạo ra các kịch bản nghiệp vụ chân thực, cho phép Trợ lý AI (LLM Agent) đưa ra cảnh báo và gợi ý nhập/xả hàng khi Demo.

3.2.4 Kỹ thuật ghi vào Cơ sở dữ liệu (Database Loading)

Sau khi làm sạch, dữ liệu được ghi vào hệ quản trị CSDL SQLite (retail.db). Do SQLite có giới hạn về số lượng biến số trong một câu lệnh SQL (thường là 999 variables), việc dùng hàm to_sql của Pandas sẽ gây lỗi OperationalError: too many SQL variables khi nạp hàng triệu dòng. Hệ thống khắc phục bằng cách:

- Sử dụng phương thức executemany() của thư viện sqlite3 kết hợp với itertuples(index=False, name=None). Việc chuyển DataFrame thành Tuple giúp tốc độ chèn (Insert) tăng gấp 10 lần so với to_sql.
- Bật các cấu hình tối ưu của SQLite (PRAGMA): journal_mode=WAL (Write-Ahead Logging) cho phép đọc và ghi song song, và cache_size=-64000 cấp phát 64MB RAM đệm, giúp quá trình nạp 59 triệu dòng lịch sử (giai đoạn 01/01/2016 — 15/08/2017) hoàn tất chỉ trong khoảng 5 phút.

Vấn đề	Quy mô	Xử lý
Giá trị thiếu oil.csv	43 ngày	Forward/Backward Fill (ffill + bfill)
Giá trị unit_sales âm	Tới -6.663	Clip về 0 (max từng item)
Giá trị cực đại	124.717 đơn vị/ngày	Giữ nguyên (đỉnh khuyến mãi)
transactions_lag1 thiếu	3.162 dòng đầu kỳ	Điền 0 an toàn

Bảng 3.1: Tóm tắt các vấn đề chất lượng dữ liệu và cách xử lý.

3.3 Feature Engineering (Kỹ thuật đặc trưng)

Trong bài toán dự báo chuỗi thời gian bán lẻ, mô hình Machine Learning (như LightGBM hay CatBoost) không thể tự động nhận diện tính mùa vụ (seasonality) hay xu hướng trễ (lag) nếu chỉ cung cấp dữ liệu giao dịch thô. Do đó, hệ thống triển khai một luồng Kỹ thuật đặc trưng (Feature Engineering) nhằm biến đổi dữ liệu chuỗi thời gian thành dạng bảng (tabular data) phẳng, giúp mô hình học được các quy luật biến động trong quá khứ để dự báo tương lai.

Việc tạo lập đặc trưng được chia làm 2 giai đoạn: Tạo đặc trưng bằng thư viện *Pandas* (vì cần xử lý Groupby và Shift theo thời gian) và Mã hóa/Điền thiếu bằng *Scikit-learn Pipeline* (để chống rò rỉ dữ liệu - Data Leakage).

3.3.1 Tạo lập Đặc trưng chuỗi thời gian (Pandas Deterministic Features)

Hệ thống xây dựng hàm `engineer_features()` chuyên trách xử lý dữ liệu trước khi đưa vào mô hình. Các nhóm đặc trưng được sinh ra bao gồm:

- **Đặc trưng Trễ (Lag Features):** Sử dụng hàm `.groupby(["store_nbr", "family"]).shift(lag)` để lấy doanh số của quá khứ. Hệ thống tạo ra 3 khoảng trễ: `sales_lag7` (nhìn lại 1 tuần), `sales_lag14` (2 tuần) và `sales_lag28` (1 tháng). Việc chọn các bội số của 7 giúp mô hình nắm bắt tính chu kỳ tuần (weekly seasonality) của siêu thị.
- **Đặc trưng Trung bình trượt (Rolling Features):** Sử dụng hàm `.transform(lambda s: s.shift(1).rolling(7).mean())` để tính trung bình động 7 ngày (`sales_rolling_mean7`). Việc áp dụng `shift(1)` bên trong `rolling` là bắt buộc để đảm bảo mô hình chỉ tính trung bình từ ngày hôm qua trở về trước, ngăn chặn hoàn toàn lỗi rò rỉ dữ liệu (Data Leakage) ở bước tạo feature.
- **Đặc trưng Thời gian & Mùa vụ (Date Features):** Trích xuất từ cột `date` thành `dayofweek` (thứ trong tuần), `month` (tháng), và biến nhị phân `is_weekend` (bằng 1 nếu là thứ 7, chủ nhật). Điều này giúp mô hình phân biệt rõ ràng hành vi mua sắm ngày thường và cuối tuần.
- **Đặc trưng Tương tác Ngày lễ (Holiday Effects):** Thay vì chỉ đánh dấu "hôm nay có phải ngày lễ không", hệ thống tạo thêm 4 biến nhị phân `is_holiday_lag1`, `lag2`, `lead1`, `lead2`. Trong đó, biến `lead` (dùng `shift(-1)`) giúp mô hình nhận diện *hiệu ứng trước lễ* (người dân đổ xô đi mua sắm tích trữ 1-2 ngày trước ngày lễ), và biến `lag` nhận diện *hiệu ứng sau lễ* (doanh số sụt giảm mạnh sau kỳ nghỉ).

3.3.2 Kỹ thuật Target Encoding và Tương tác Cụm cửa hàng (Cluster Features)

Đặc trưng `cluster` (cụm cửa hàng có hành vi mua sắm tương tự) mang tính phân hóa rất cao. Để khai thác tối đa, hệ thống xây dựng class `ClusterFeatureEngineer` để thực hiện Lagged Target Encoding:

- **Thông kê trễ (Lagged Statistics):** Tính toán `cluster_mean_sales` và `cluster_std_sales` dựa trên trung bình và độ lệch chuẩn của doanh số toàn cụm trong 28 ngày qua. Quá trình này sử dụng `shift(1).rolling(28)` để đảm bảo không sử dụng dữ liệu tương lai.
- **Tương tác đa biến (Interaction Features):** Tạo cột `cluster_promo_lift` bằng cách tính tỷ lệ giữa doanh số trung bình khi có khuyến mãi và không có khuyến mãi của cụm, đo lường *mức độ nhạy cảm với khuyến mãi* của từng nhóm cửa hàng.
- **Đánh dấu Cụm chủ lực (Tier Flag):** Tạo biến nhị phân `is_tier1_cluster` nhận giá trị 1 nếu cửa hàng thuộc Cluster 5 hoặc 11 (các siêu thị lớn, quy mô cao). Điều này giúp mô hình Tree-based tự động chia nhánh dữ liệu theo cấp độ cửa hàng.

3.3.3 Mã hóa Biến Phân loại và Xử lý Thiếu dữ liệu (Scikit-learn Pipeline)

Vì các đặc trưng Lag/Rolling luôn bị thiếu (NaN) ở những ngày đầu tiên của chuỗi thời gian, và các biến phân loại chưa được mã hóa số, hệ thống xây dựng hàm `build_preprocessor()` sử dụng `ColumnTransformer` của Scikit-learn. Lớp này được `fit` trên tập Train và `transform` trên tập Validation/Test (chống leakage thống kê):

- **Nhánh Biến số (Numeric):** Các cột `sales_lag7`, `transactions_lag1`, `cluster_mean_sales` được đưa qua `SimpleImputer(strategy="constant", fill_value=0)`. Việc điền 0 cho các ngày đầu chuỗi mang ý nghĩa nghiệp vụ là "chưa có giao dịch trước đó".
- **Nhánh Biến Phân loại (Categorical):** Các cột `store_nbr`, `family`, `city`, `state`, `holiday_type` được mã hóa bằng `OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)`. Kỹ thuật này cho phép mô hình xử lý các danh mục mới chưa từng xuất hiện ở tập Train (ví dụ thành phố mới) bằng cách gán bằng -1, thay vì văng lỗi.
- **Nhánh Đi qua thẳng (Passthrough):** Các biến nhị phân (`is_weekend`, `is_holiday`) và biến liên tục (`oil_price`) được giữ nguyên giá trị đưa vào mô hình.

3.3.4 Biến đổi Biến Mục tiêu (Target Transformation)

Do metric đánh giá của cuộc thi là RMSLE (Root Mean Squared Logarithmic Error), sai số bị phạt rất nặng nếu mô hình dự báo một giá trị quá nhỏ so với giá trị thực tế lớn. Để mô hình học đồng đều hơn, hệ thống áp dụng phép biến đổi $\log(1 + x)$ cho biến mục tiêu

(target) trong quá trình huấn luyện bằng hàm `np.log1p()`. Kết quả dự báo đầu ra từ mô hình sẽ được chuyển ngược về thang đo thực tế bằng hàm `np.expm1()`, sau đó mới được dùng để tính toán RMSLE hoặc đưa vào CSDL.

Bảng 3.2 dưới đây tổng hợp toàn bộ các bộ đặc trưng được hệ thống tự động sinh ra.

Bảng 3.2: Bộ đặc trưng sử dụng trong mô hình dự báo.

Nhóm	Đặc trưng	Mô tả
Lag (Trễ)	sales_lag7	Doanh số của cùng (store, family) cách đây 7 ngày.
	sales_lag14	Doanh số cách đây 14 ngày.
	sales_lag28	Doanh số cách đây 28 ngày.
Rolling (Trượt)	sales_rolling_mean7	Trung bình động 7 ngày. Dùng <i>shift(1)</i> để chỉ lấy dữ liệu quá khứ, chống leakage.
Lịch / Mùa vụ	dayofweek	Thứ trong tuần (0 đến 6).
	month	Tháng trong năm.
	is_weekend	Biến nhị phân (0/1), bằng 1 nếu Thứ 7 hoặc Chủ Nhật.
Ngày lễ	is_holiday	Biến nhị phân cho biết ngày hiện tại có phải ngày lễ không.
	is_holiday_lag1	Hiệu ứng sau lễ: Hôm qua có phải ngày lễ không.
	is_holiday_lag2	Hiệu ứng sau lễ 2 ngày.
	is_holiday_lead1	Hiệu ứng trước lễ: Ngày mai có phải ngày lễ không.
	is_holiday_lead2	Hiệu ứng trước lễ 2 ngày.
Cluster (Cụm CH)	cluster_mean_sales	Target Encoding: Doanh số trung bình của cụm cửa hàng trong 28 ngày qua.
	cluster_promo_lift	Tương tác đa biến: Mức độ nhạy cảm khuyến mãi của từng cụm.
	is_tier1_cluster	Biến nhị phân = 1 nếu thuộc cụm cửa hàng chủ lực (Cluster 5, 11).
Ngoại sinh	oil_price	Giá dầu mỏ của ngày hiện tại (đã điền dữ liệu thiếu cuối tuần).
Phân loại	store_nbr, family, city...	Các biến danh mục được mã hóa bằng <i>OrdinalEncoder</i> .

3.4 Huấn luyện Mô hình và Tích hợp Hệ thống

3.4.1 Huấn luyện Global Ensemble (LightGBM + CatBoost)

Để xây dựng mô hình dự báo toàn cục (Global Model), hệ thống kết hợp hai thuật toán tiên tiến nhất cho dữ liệu bảng (tabular data) là LightGBM và CatBoost. Biến mục tiêu (target) được biến đổi bằng hàm $\log(1 + y)$ trước khi huấn luyện để đảm bảo tính ổn định của phương sai, và quá trình đánh giá được thực hiện trong không gian log bằng metric RMSLE.

Hệ thống áp dụng phương pháp **Kiểm định Walk-forward 3-fold** nhằm mô phỏng chính xác môi trường dự báo thực tế. Mỗi fold sử dụng 28 ngày cuối cùng làm tập xác thực (Validation set), tương ứng với các ngày kết thúc: 20/06/2017, 18/07/2017 và 15/08/2017. Tổng dữ liệu huấn luyện tối thiểu được giữ ở mức 365 ngày để đảm bảo mô hình nắm bắt được tính mùa vụ hàng năm.

Siêu tham số

Bảng 3.3 trình bày các siêu tham số (hyperparameters) chính được tinh chọn cho hai mô hình Global.

Bảng 3.3: Siêu tham số chính của hai mô hình Global.

LightGBM		CatBoost	
Tham số	Giá trị	Tham số	Giá trị
n_estimators	3000	iterations	3000
learning_rate	0.03	learning_rate	0.03
num_leaves	63	depth	8
subsample	0.8	l2_leaf_reg	3.0
colsample_bytree	0.8	loss_function	RMSE
reg_alpha	0.1	—	—
reg_lambda	0.1	—	—

Tối ưu trọng số hòa trộn

Để tìm ra tỷ lệ kết hợp (blend) tối ưu giữa LightGBM và CatBoost, hệ thống sử dụng phương pháp tìm kiếm một chiều (`scipy.optimize.minimize_scalar`) trên đoạn $[0, 1]$ nhằm cực tiểu hóa RMSLE của kết quả hòa trộn trên các fold validation:

$$w^* = \arg \min_{w \in [0,1]} \text{RMSLE} (w \cdot \hat{y}_{LGBM} + (1 - w) \cdot \hat{y}_{CatBoost}) \quad (3.1)$$

Kết quả tối ưu thu được là $w_{LGBM} = 0.4163$ và $w_{CatBoost} = 0.5837$. Số vòng lặp tốt nhất trung bình (*best_iteration*) qua các fold là 2.511 đối với LightGBM và 2.266 đối với CatBoost.

3.4.2 Huấn luyện 33 Local Models

Thay vì chỉ dùng một mô hình chung cho toàn bộ hệ thống, hệ thống tiến hành huấn luyện riêng một mô hình LightGBM cho từng ngành hàng (*family*) với chiến lược thích ứng (adaptive strategy) dựa trên dung lượng dữ liệu:

- **Cấu hình cơ bản:** Sử dụng `learning_rate = 0.05`, `num_leaves = 31`.
- **Dữ liệu nhỏ (< 5.000 dòng):** Thu nhỏ kiến trúc mạng (`num_leaves = 15`, `min_child_samples = 5`) để chống overfitting.
- **Dữ liệu quá nhỏ (< 1.000 dòng):** Bỏ qua việc train mô hình riêng, luôn chuyển sang dùng tuyến Global.

Quá trình huấn luyện sử dụng kỹ thuật Early Stopping để chọn ra *best_iteration*, sau đó tiến hành huấn luyện lại (Retrain) trên toàn bộ dữ liệu của ngành hàng đó với ngân sách vòng lặp tăng thêm 10% (`best_iter × 1.1`) nhằm tối đa hóa khả năng học.

Kết quả: 33/33 ngành hàng đều đủ điều kiện huấn luyện, cho ra `artifact local_lgbm_models.pkl` (kích thước ≈ 6.8 MB).

3.4.3 Dự báo đệ quy 16 ngày và Smart Routing

Quá trình suy luận (Inference) chạy đệ quy (recursive) từng ngày trong khoảng từ 16/08 đến 31/08/2017. Tại mỗi bước dự báo, hệ thống tái sinh các đặc trưng Lag và Rolling dựa trên chuỗi dữ liệu đã bao gồm các kết quả dự báo vừa sinh ra ở các ngày trước đó (prediction feedback), đồng thời bảo toàn nguyên vẹn các metadata tĩnh (city, state, type, cluster, cluster_family_id, perishable).

Logic Smart Routing

Hệ thống định tuyến thông minh luồng dự báo theo các bước sau:

1. Nếu ngành hàng có Local Model: Dự báo tuyến chính bằng Local LGBM, sau đó nghịch log bằng hàm $e^x - 1$ (`expm1`).
2. Ngược lại (Fallback global ensemble): Tính toán $\hat{y} = w\hat{y}_{LGBM} + (1 - w)\hat{y}_{Cat}$, và ghi nhận metadata `used_model = Global Ensemble`.
3. Toàn bộ kết quả âm bị clip về 0 để đảm bảo tính hợp lệ của doanh số.

Phân rã xuống cấp SKU

Dự báo ở cấp ngành hàng (*family*) được phân rã xuống cấp độ mã sản phẩm (SKU) theo tỷ lệ chia sẻ (share) được tính từ 45 ngày lịch sử gần nhất kết thúc vào 15/08/2017:

$$\hat{y}_{SKU,t} = \hat{y}_{family,t} \times \frac{\sum_{d=1}^{45} y_{SKU,t-d}}{\sum_{d=1}^{45} y_{family,t-d}} \quad (3.2)$$

Toàn bộ kết quả được nạp vào bảng `forecasts` trong CSDL, bao gồm 3.370.464 dòng (tương đương $16 \text{ ngày} \times 54 \text{ cửa hàng} \times \approx 1.900 \text{ SKU/cửa hàng}$).

3.4.4 Chuẩn bị submission Kaggle

Hai tệp nộp bài (submission) được tạo ra từ hai chiến lược độc lập để so sánh hiệu năng:

1. `submission_local.csv`: Dự báo thuần túy từ 33 Local LightGBM Models.
2. `submission_ensemble.csv`: Dự báo từ Global Ensemble (LightGBM 41.63% + CatBoost 58.37% blend).

Cả hai tệp có cấu trúc chuẩn Kaggle (`id = store_nbr_item_nbr_onpromotion_date, unit_sales`), trải dài trong 16 ngày test từ 16/08 đến 31/08/2017.

3.5 Khởi tạo cơ sở dữ liệu và tích hợp triển khai

3.5.1 Script khởi tạo

Hệ thống sử dụng các script Python tự động hóa để dựng môi trường dữ liệu:

- `init_database.py`: Dựng schema, chỉ mục, nạp dữ liệu lịch sử theo khối (chunking + `executemany`), mô phỏng tồn kho (60% bình thường, 20% sắp hết, 20% động vốn; lead time 1–7 ngày) và dựng hai bảng tổng hợp cache.
- `init_auth.py`: Nạp tài khoản xác thực gồm Admin, Tenant Manager, Store Manager.
- `build_sales_cache.py`: Dựng AGG tables với cấu trúc WITHOUT ROWID, WAL mode, `synchronous=NORMAL`.
- `build_business_cache.py`: Dựng lại cache nghiệp vụ khi có thay đổi.

3.5.2 Docker Compose deployment

Hệ thống được triển khai bằng `docker-compose` với các cấu hình hạ tầng sau:

- Chuỗi healthcheck đảm bảo thứ tự khởi động: `ml_service` → `backend` → `frontend`.
- Nginx reverse proxy: Điều hướng `/api` về `backend:8000` (timeout 300s), bật gzip và `no-cache` cho SPA.

- Mạng bridge riêng `retail_network` để cô lập và bảo mật giao tiếp nội bộ.
- Volume mount trọng số mô hình ở chế độ read-only để bảo vệ tính toàn vẹn của file model.

3.5.3 Vận hành

Hệ thống cho phép thay đổi linh hoạt mô hình ngôn ngữ (LLM) thông qua biến môi trường `LLM_MODEL_NAME` (Ví dụ: do qwen/qwen3-32b bị Groq ngừng phục vụ, hệ thống đã chuyển sang qwen/qwen3.6-27b). Ngoài ra, biến `GROQ_BASE_URL` cho phép trỏ client sang endpoint OpenAI-compatible khác (ví dụ OrcaRouter) khi cần, còn `LLM_REASONING_EFFORT` kiểm soát mức suy luận (thinking) của mô hình Qwen 3. Sau khi đổi file `.env`, cần recreate container backend bằng lệnh:

```
docker compose up -d --force-recreate backend
```

CHƯƠNG 4

KẾT QUẢ, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Kết quả huấn luyện mô hình (Validation nội bộ)

4.1.1 Global Ensemble — Kiểm định Walk-Forward

Bộ chỉ số đầy đủ (MAE, RMSE, WAPE, WMAPE, R^2 , kèm RMSLE để đối chiếu) được tính bởi script chuyên biệt `evaluate_metrics.py` với giao thức walk-forward 3 fold, mỗi fold validation 28 ngày (cùng điểm cắt với quá trình huấn luyện), blend LightGBM + CatBoost thực hiện trong không gian log; toàn bộ số liệu lưu tại `ml_training/models/evaluation_metrics.json`.

Fold (kỳ validation)	MAE	RMSE	WAPE	WMAPE	RMSLE	R^2	w_{LGBM}
1 (24/05–20/06/2017)	74.57	278.27	13.02%	12.75%	0.35429	0.9647	0.7002
2 (21/06–18/07/2017)	66.85	248.99	12.00%	11.69%	0.34359	0.9691	0.8382
3 (19/07–15/08/2017)	76.22	246.20	13.83%	13.48%	0.36181	0.9672	0.9939
Pooled (gộp 3 fold)	72.54	258.28	12.95%	12.63%	0.35330	0.9669	—

Bảng 4.1: Kết quả kiểm định walk-forward của Global Blend trên tập validation nội bộ (MAE/RMSE/WAPE/WMAPE/RMSLE: thấp hơn là tốt hơn; R^2 càng gần 1 càng tốt).

Mô hình	MAE	RMSE	WAPE	WMAPE	RMSLE	R^2
LightGBM (global)	72.52	257.89	12.95%	12.63%	0.35353	0.9670
CatBoost (global)	76.37	272.31	13.63%	13.36%	0.35857	0.9632
Blend (hòa trộn log-space)	72.54	258.28	12.95%	12.63%	0.35330	0.9669

Bảng 4.2: So sánh chỉ số pooled (gộp toàn bộ 3 fold) giữa hai mô hình thành phần và blend.

Đọc kết quả theo bộ chỉ số đa tiêu chí

- **WAPE 12.95% (pooled)**: tương đương Forecast Accuracy $\approx 87\%$ — trung bình mỗi điểm dự báo (cửa hàng $\times$ ngành hàng $\times$ ngày) lệch khoảng 13% tổng doanh số thực tế, mức mà hệ thống có thể truyền thẳng cho người dùng cuối trên nền tảng;

- **WMAPE 12.63%** thấp hơn WAPE 12.95%: với trọng số 1.5 cho hàng dễ hỏng, sai số tương đối của nhóm “độ tươi” thấp hơn mức trung bình — mô hình làm việc tốt chính ở nhóm có rủi ro đứt hàng cao nhất;
- $R^2 = 0.9669$: mô hình giải thích được khoảng 96.7% phương sai doanh số trên tập validation, ổn định trên cả 3 fold (0.9647 – 0.9691);
- **MAE 72.54 vs RMSE 258.28**: RMSE gấp ≈ 3.6 lần MAE, xác nhận tồn tại đuôi sai số lớn tập trung ở nhóm tốc độ bán cao (GROCERY I, BEVERAGES) — nhóm này được giám sát riêng bằng RMSE;
- **Blend so với LightGBM thuần**: blend chỉ nhỉnh hơn về RMSLE (0.35330 vs 0.35353) và gần như ngang bằng các chỉ số còn lại — việc chọn blend là để tối ưu metric mục tiêu trong không gian log, trong khi CatBoost đóng vai trò “bảo hiểm” ở các fold đầu ($w_{\text{LGBM}} = 0.70$ ở fold 1);
- **Xu hướng trọng số**: w_{LGBM} tăng dần $0.70 \rightarrow 0.84 \rightarrow 0.99$ — optimizer gần như chọn thuần LightGBM ở fold cuối, phù hợp với việc LightGBM thắng CatBoost trên mọi chỉ số pooled;
- **Fold 2 tốt nhất** (21/06–18/07/2017: WAPE 12.00%, RMSLE 0.34359); **fold 3 khó nhất** (19/07–15/08/2017: WAPE 13.83%) do rơi vào mùa tựu trường và giai đoạn biến động cao ngay trước kỳ dự báo Kaggle.

4.1.2 Ảnh hưởng của điểm số đến các quyết định sau huấn luyện

Điểm số không chỉ là thước đo ghi lại ở cuối quy trình — trong hệ thống, chúng đóng vai trò **tín hiệu điều khiển** cho năm “cổng quyết định” diễn ra ngay sau khi mô hình hoàn tất huấn luyện. Mỗi cổng đọc một điểm số khác nhau và dẫn tới một quyết định tự động khác nhau; hiểu đúng vai trò của từng cổng là điều kiện để đọc kết quả validation ở hai bảng trên một cách chính xác.

Cổng 1 — Điểm số quyết định độ dài huấn luyện (early stopping)

Do mục tiêu tối ưu là RMSLE trên không gian log, biến mục tiêu được biến đổi bằng $\log(1 + y)$ (phần Biến đổi Biến Mục tiêu) và quá trình dừng sớm theo dõi chỉ số RMSE tính trên biến mục tiêu log-space của tập kiểm định, với kiên nhẫn 100 vòng. Điểm validation sau mỗi vòng — chứ không phải số vòng lặp cấu hình ban đầu — quyết định độ dài huấn luyện thực tế: LightGBM dừng ở 1.013 / 1.888 / 1.699 vòng và CatBoost ở 3.498 / 1.051 / 1.258 vòng qua ba fold (siêu dữ liệu `ensemble_meta.json`), thay vì chạy hết ngưỡng cấu hình. Với 33 local model, số vòng tốt nhất còn được dùng để retrain trên toàn bộ dữ liệu với hệ số đệm 1.1 lần — nói cách khác, điểm số trực tiếp “viết lại” siêu tham số số vòng lặp của mô hình.

Cổng 2 — Điểm số quyết định cấu trúc ensemble

Trọng số hòa trộn không được đặt tay mà được tối ưu bằng `minimize_scalar` trên đoạn $[0, 1]$ với hàm mất mát là RMSLE trong không gian log (phần Tối ưu trọng số hòa trộn), kèm một rào chắn: nghiệm trong đoạn không được tệ hơn mô hình đơn tốt nhất, nếu không hệ thống quay về đầu mút $w \in \{0, 1\}$. Kết quả theo từng fold trong bảng 4.1 ($w_{\text{LGBM}} = 0.70 \rightarrow 0.84 \rightarrow 0.99$) cho thấy đây là một quyết định “mở”: ensemble được tái cấu hình theo điểm số của từng đợt kiểm định thay vì cố định một tỷ lệ hòa trộn.

Cổng 3 — Ngưỡng $\geq 1\%$ quyết định giữ hay bỏ siêu tham số mới

Mỗi vòng tìm kiếm siêu tham số phải vượt một cổng chấp nhận: bộ tham số mới chỉ được cập nhật khi RMSLE blend trên fold tốt hơn baseline ít nhất 1% (`OPTUNA_MIN_IMPROVEMENT`). Nếu không đạt ngưỡng, hệ thống **giữ nguyên tham số cũ**; kết quả của lần thử (baseline, best, phần trăm cải thiện, cờ accepted) vẫn được ghi vào `ensemble_meta.json` để truy vết. Cổng này phản ánh một nguyên tắc chống overfit siêu tham số: một cải thiện nhỏ hơn 1% nhiều khả năng là nhiễu của fold chứ không phải tín hiệu tổng quát, và việc đổi tham số theo nhiễu còn nguy hiểm hơn việc giữ một cấu hình đã được kiểm chứng.

Cổng 4 — Tách “điểm để chọn” khỏi “điểm để báo cáo”

Điểm blend trong bảng 4.1 là điểm *fold-tuned*: trọng số w và vòng dừng sớm đều được chọn trên chính tập kiểm định của fold, nên mang tính lạc quan. Để đối trọng, pipeline tính thêm điểm **LOO** (leave-one-out): trọng số dùng để chấm fold i chỉ được tối ưu trên các fold còn lại. Quy ước của hệ thống là phân tách rõ hai loại điểm — điểm *fold-tuned* dùng để *chọn* cấu hình, điểm LOO trung thực dùng để *báo cáo* — nhằm tránh tình trạng tự đánh lừa bằng một điểm số được tối ưu trên chính tập đo lường nó.

Cổng 5 — Điểm số quyết định chiến lược phục vụ

Sau huấn luyện, mô hình được đo theo hai chế độ phục vụ: dự báo làm mới hằng ngày (*recursive*, phản ánh đúng cách hệ thống vận hành) đạt RMSLE 0.35168, trong khi dự báo một lần cho cả kỳ 28 ngày (*one-shot*) đạt 0.38575 — suy giảm +9.7%. Sự suy giảm còn theo độ xa của horizon: tuần 1–4 lần lượt là $0.353 \rightarrow 0.386 \rightarrow 0.394 \rightarrow 0.407$ (siêu dữ liệu `ensemble_meta.json`, trường `product_quality`). Điểm số theo horizon vì vậy trực tiếp định hình kiến trúc phục vụ: hệ thống chọn re-forecast hằng ngày làm chế độ mặc định, one-shot chỉ sử dụng khi bắt buộc, và các nhận định cho tuần 3–4 được đánh dấu là cần thận trọng.

Cổng	Điểm số được đọc	Quyết định tự động	Hệ quả
Early stopping	RMSE log-space trên val	Số vòng huấn luyện thực tế	1.013 – 3.498 vòng/fo
Trọng số blend	RMSLE log-space (argmin)	w_{LGBM} theo từng fold	0.70 → 0.99
Cổng siêu tham số	RMSLE cải thiện $\geq 1\%$	Cập nhật hay giữ tham số	Chống overfit nhiều fo
Điểm báo cáo	LOO trung thực	Tách điểm chọn / điểm báo cáo	Tránh tự đánh lừa
Chiến lược phục vụ	RMSLE theo chế độ + horizon	Recursive là mặc định	One-shot thua +9.7%

Bảng 4.3: Tổng hợp các cổng quyết định do điểm số điều khiển sau khi huấn luyện.

Tóm lại, cùng một mô hình sau huấn luyện được nhìn qua ba lớp điểm số với ba mục đích khác nhau: điểm **tối ưu** (RMSLE không gian log) điều khiển biến đổi mục tiêu, độ dài huấn luyện và trọng số blend; điểm **báo cáo** (WAPE, WMAPE, R^2 — như đã phân tích ở phần trên và bảng 4.2) trả lời câu hỏi nghiệp vụ “mô hình đủ dùng chưa”; và điểm **kiểm chứng ngoài** (NWRMSLE trên Kaggle, phần Kết quả trên Kaggle Leaderboard) xác nhận khả năng tổng quát hóa trên dữ liệu chưa từng thấy. Một quyết định sau huấn luyện đúng đắn đòi hỏi dùng đúng điểm số cho đúng mục đích — đọc điểm tối ưu để báo cáo nghiệp vụ, hoặc ngược lại, sẽ dẫn tới kết luận sai lệch cả hai phía.

4.1.3 Local Models — 33 ngành hàng

Bảng 4.4: RMSLE kiểm định của 33 mô hình local theo ngành hàng (tập validation nội bộ).

Ngành hàng	Số dòng	Best iter	RMSLE
PRODUCE	31.208	180	0.14241
BOOKS	2.772	128	0.14650
DAIRY	31.208	211	0.15117
BREAD/BAKERY	31.208	82	0.16923
GROCERY I	31.208	244	0.16947
DELI	31.207	89	0.18699
MEATS	31.207	389	0.20492
POULTRY	31.207	104	0.21073
BEVERAGES	31.208	554	0.21539
HOME CARE	31.207	70	0.23973
HOME APPLIANCES	7.919	171	0.24572
PERSONAL CARE	31.207	84	0.24919
BABY CARE	4.491	45	0.26393
FROZEN FOODS	31.205	206	0.26647
PREPARED FOODS	31.202	257	0.27197

Ngành hàng	Số dòng	Best iter	RMSLE
EGGS	31.207	255	0.29911
CLEANING	31.208	550	0.30796
PET SUPPLIES	25.881	277	0.39148
HARDWARE	18.483	67	0.40054
SEAFOOD	28.541	87	0.41154
MAGAZINES	26.308	628	0.41371
BEAUTY	26.204	85	0.42245
PLAYERS AND ELECTRONICS	30.065	377	0.42357
LAWN AND GARDEN	19.741	80	0.43471
LIQUOR,WINE,BEER	29.953	203	0.44684
AUTOMOTIVE	30.214	137	0.45607
HOME AND KITCHEN I	31.157	62	0.45808
HOME AND KITCHEN II	31.110	67	0.46853
LADIESWEAR	21.485	105	0.48061
CELEBRATION	30.476	267	0.49956
GROCERY II	29.588	94	0.52180
LINGERIE	27.428	166	0.54474
SCHOOL AND OFFICE SUPPLIES	15.339	179	0.78876

4.1.4 Phân tích kết quả validation

Nhóm hàng tiêu dùng thiết yếu (PRODUCE, DAIRY, BREAD/BAKERY, GROCERY I) đạt RMSLE 0.14–0.17 — rất phù hợp smart routing tuyến chính.

Nhóm biến động mạnh theo mùa học tập (SCHOOL AND OFFICE SUPPLIES 0.79) có sai số cao hơn, đúng dự đoán từ lý thuyết bias–variance: local model kém ổn định ở chuỗi nhiễu, hệ thống vẫn bảo đảm fallback global khi cần.

4.2 Kết quả trên Kaggle Leaderboard

4.2.1 Kết quả submission thực tế

Hệ thống đã submit hai phương án dự báo lên cuộc thi Kaggle *Favorita Grocery Sales Forecasting*. Chỉ số đánh giá là **NWRMSLE** (Normalized Weighted RMSLE — thấp hơn là tốt hơn):

4.2.2 Phân tích kết quả Kaggle

Submission	Public Score	Private Score	Gap	Trạng thái
submission_ensemble.csv	0.64438	0.63542	-0.00896	Best Score
submission_local.csv	0.65307	0.64207	-0.01100	Complete
Cải thiện bởi Ensemble	-0.00869	-0.00665		

Bảng 4.5: Kết quả submission trên Kaggle Favorita Grocery Sales Forecasting (NWRM-SLE — thấp hơn là tốt hơn).

Global Ensemble vượt trội Local Models

Submission sử dụng Global Ensemble (LightGBM + CatBoost blend) đạt score tốt hơn so với chỉ dùng Local Models trên cả hai leaderboard: Public 0.64438 vs 0.65307 (cải thiện **0.00869** điểm) và Private 0.63542 vs 0.64207 (cải thiện **0.00665** điểm).

Kết quả xác nhận giá trị của chiến lược kết hợp mô hình toàn cục (Global) với chuyên biệt hóa cục bộ (Local): Global Ensemble có khả năng “bình quân hóa” các outlier, giúp tổng thể ổn định hơn so với 33 local model riêng lẻ vốn nhạy cảm với nhiễu của từng ngành hàng.

Private Score tốt hơn Public Score — Dấu hiệu tích cực

Cả hai submission đều có private score tốt hơn public score (Ensemble: $0.63542 < 0.64438$, gap -0.00896; Local: $0.64207 < 0.65307$, gap -0.01100). Đây là tín hiệu **tích cực** cho thấy mô hình có khả năng tổng quát hóa (generalization) tốt trên dữ liệu chưa từng thấy, không hiện tượng overfit vào public test set — điều thường gặp khi người tham gia lặp đi lặp lại thử nghiệm theo public leaderboard.

Phân tích khoảng cách validation nội bộ — Kaggle test

Chỉ số	Validation nội bộ	Kaggle Test	Gap
Global Ensemble	0.35330 (RMSLE)	0.64438 (NWRMSLE)	+0.29108
Local Models (TB)	≈ 0.35 (RMSLE)	0.65307 (NWRMSLE)	+0.30307

Bảng 4.6: So sánh kết quả validation nội bộ và test trên Kaggle.

Nguyên nhân khoảng cách:

1. **Khác biệt chỉ số:** Validation nội bộ dùng RMSLE thuần túy, trong khi Kaggle dùng NWRMSLE có trọng số 1.25 cho ngành hàng dễ hỏng (perishable) làm tăng score tổng thể;

2. **Khác biệt cấp độ dự báo:** Validation ở cấp (store $\times$ family) đã được tổng hợp, submission Kaggle phải phân rã xuống cấp (store $\times$ item) — sai số tích lũy qua bước phân rã theo tỷ lệ chia sẻ 45 ngày;
3. **Khác biệt phân phối dữ liệu:** Tập validation walk-forward dùng dữ liệu 05/2017–08/2017 đã biết trước cho feature lag, trong khi dự báo đệ quy 16 ngày trên Kaggle phải sinh lag features từ chính các dự báo trung gian (prediction feedback) làm sai số lan truyền;
4. **Yếu tố bất thường:** Giai đoạn test 16/08—31/08/2017 có biến động mùa tựu trường, sau động đất Ecuador và các biến động kinh tế vĩ mô khó dự đoán từ dữ liệu huấn luyện.

4.2.3 So sánh với baseline và top solution

Phương pháp	Public	Private	Ghi chú
Naive (lag-7)	≈ 0.85	≈ 0.83	Baseline tham chiếu
Seasonal Naive	≈ 0.75	≈ 0.74	Baseline tham chiếu
Local Models (đề tài)	0.65307	0.64207	33 LightGBM chuyên biệt
Global Ensemble (đề tài)	0.64438	0.63542	Best Score — LGBM + CatBoost
Top solution Kaggle 2018	≈ 0.36	≈ 0.35	Đội vô địch (ensemble phức tạp)

Bảng 4.7: So sánh kết quả hệ thống với baseline và top solution Kaggle Favorita.

4.2.4 Đánh giá vị trí trên leaderboard

Với private score 0.63542 (Ensemble), hệ thống đạt kết quả ở mức tương đương nhóm giữa leaderboard Kaggle Favorita Grocery Sales Forecasting. So với đội vô địch (NWRMSLE ≈ 0.35), khoảng cách 0.28 điểm phản ánh các giới hạn của một dự án capstone học tập:

- **Chỉ dùng Gradient Boosting:** Không kết hợp Deep Learning (N-BEATS, DeepAR, TFT) vốn thể hiện mạnh ở top leaderboard;
- **Feature engineering cơ bản:** Chưa khai thác Fourier terms, target encoding cấp SKU, external data (thời tiết, demographics);
- **Không dùng multi-seed ensemble:** Top solution thường blend 10–20 model với seed khác nhau;
- **Giới hạn tính toán:** Huấn luyện CPU, không GPU cluster cho hyperparameter search rộng.

Tuy vậy, với bối cảnh là dự án học tập trong khuôn khổ Samsung Innovation Campus AI Course, kết quả NWRMSLE 0.63542 là thành tích đáng ghi nhận, chứng minh pipeline end-to-end hoạt động đúng đắn và kiến trúc Global Ensemble + Smart Routing có giá trị

thực tiễn.

4.3 Nhật ký thí nghiệm và bảng chứng vận hành

Phần này tổng hợp log gốc và kết quả đo lường của lần chạy thí nghiệm cải tiến mô hình (01–02/09/2026; Python 3.14.6, LightGBM 4.7, CatBoost 1.2.10, scikit-learn 1.9), lưu tại thư mục `report_logs/`. Mọi con số được trích trực tiếp từ log — không suy diễn. Đây đồng thời là case study cụ thể của cổng chấp nhận siêu tham số đã mô tả ở mục “Ảnh hưởng của điểm số đến các quyết định sau huấn luyện”.

4.3.1 Timeline thí nghiệm

Thời điểm	Sự kiện
02/09, 00:30	Backup toàn bộ model artifacts cũ → <code>models_backup_20260902/</code> (61MB)
02/09, 00:40	Train global: Optuna 24 trial (fold-3) → 3 fold walk-forward → retrain 936.506 dòng (5,5 giờ)
02/09, 06:15	Train xong. Cổng chấp nhận FAIL → rollback artifacts về bản base-line
04/09, 09:38	Chạy <code>predict.py</code> → <code>submission_ensemble.csv</code> 3.370.464 dòng (4.5 phút)

Bảng 4.8: Timeline lần thí nghiệm cải tiến mô hình 01–02/09/2026.

4.3.2 Kết quả baseline so với thí nghiệm

Cùng giao thức: walk-forward 3 fold × 28 ngày, feature spec v2.1 (65 cột), ES trên validation, blend log-space tối ưu w . Nguồn: `ensemble_meta_baseline.json`, `ensemble_meta_experiment.json`

Chỉ số	Baseline (đang chạy)	Thí nghiệm	Chênh
Blend RMSLE (fold-tuned)	0.35168	0.37913 ± 0.0056	tệ hơn 7.8%
— Fold 1 (val 24/05→20/06)	—	0.38298	
— Fold 2 (val 21/06→18/07)	—	0.37123	
— Fold 3 (val 19/07→15/08)	—	0.38319	
One-shot 28 ngày	0.38575 (+9.7%)	0.4162 (+9.8%)	tệ hơn
Blend weight (w_{LGBM})	0.9359 (median)	0.610 (oof)	

Bảng 4.9: So sánh baseline đang phục vụ với bản thí nghiệm.

Vòng Optuna 24 trial trên fold-3 cho kết quả baseline blend 0.38329 → best 0.38261 — chỉ cải thiện +0.18%, dưới ngưỡng chấp nhận 1%, nên hệ thống tự giữ tham số cũ;

đồng thời vì điểm tổng thể của bản thí nghiệm tệ hơn baseline, artifacts được rollback. Bộ levers của lần thí nghiệm gồm $N_SEEDS=2$, recency weighting với half-life 180 ngày và feature mới `sales_std7/std28` kèm lịch đầy đủ (`payday`, `dayofmonth`, `weekofmonth`, `dayofyear`, `is_month_start/end`). Kết luận rút ra: recency weighting 180 ngày là nghi phạm chính làm điểm xấu đi; feature mới không mang tín hiệu đủ mạnh (Optuna không vượt được baseline).

4.3.3 Bộ sáu chỉ số của bản thí nghiệm

Trên pooled OOF của bản thí nghiệm (nguồn: `ensemble_meta_experiment.json`):

Model	RMSLE	MAE	RMSE	WAPE	WMAPE	R^2
LGBM	0.37985	57.61	220.37	11.56%	11.33%	0.97337
CatBoost	0.38082	56.81	216.10	11.40%	11.19%	0.97440
Blend	0.37919	56.61	216.05	11.36%	11.14%	0.97441

Bảng 4.10: Sáu chỉ số pooled OOF của bản thí nghiệm (phân biệt với bảng pooled của bản đang phục vụ ở bảng 4.2).

NWRMSLE (chuẩn chấm Kaggle, $w = 1.25$ cho perishable) của bản thí nghiệm: 0.37176.

4.3.4 Sinh submission và kiểm thử

Sinh submission (04/09): `python ml_training/src/predict.py` ở chế độ recursive, `min_lag=1`, $w_{LGBM} = 0.9359$ (từ ensemble baseline) sinh `submission_ensemble.csv` gồm 3.370.464 dòng (72.5% khác 0) trong 4.5 phút. Hai lỗi đã được vá trong `predict.py` qua lần chạy này: (1) dtype nullable cho cột `onpromotion` khi đọc CSV theo chunk (pandas 3.0 Dtype-Warning); (2) gộp nhóm trùng ở ranh giới chunk trước khi dựng `complete_panel`.

Kết quả kiểm thử: `pytest -m 'not llm and not slow'` cho **130 passed / 1 failed / 1 skipped** trong 2 phút 19 giây; fail duy nhất là `test_performance_all_endpoints` — P95 endpoint Products 1369,6ms vượt SLA 1000ms trên máy chạy cục bộ (vấn đề độ trễ phần cứng, không phải lỗi đúng đắn). Riêng bộ 12 unit test chỉ số và feature contract (`test_train_metrics.py`) đạt 12/12: công thức MAE/RMSE/WAPE/WMAPE/ R^2 /RMSLE khớp chuẩn, rolling std đúng `shift(1).rolling(7)`, `payday` đúng, không leak target vào lag/rolling. Bộ test LLM chat (`llm`, `slow`) chưa chạy trong đợt này do cần quota Groq (free 200k token/ngày).

4.3.5 Tái tạo các log

Log	Lệnh
train_global_20260902.log	python ml_training/src/train.py (5.5h, cần train.csv 4.7GB)
predict_run_20260904.log	python ml_training/src/predict.py (5 phút)
tests_run.log	python -m pytest -m 'not llm and not slow' -q
tests_train_metrics.log	pytest backend/tests/test_train_metrics.py -q

Bảng 4.11: Lệnh tái tạo từng log trong thư mục report_logs/.

4.4 Kiểm thử API Endpoints

Các kết quả trong mục này được thực thi qua bộ kiểm thử tự động **pytest** đặt tại backend/tests/, gồm sáu module: test_api.py (hành vi endpoint), test_security.py (JWT, Row-Level Isolation, SQL injection, truy cập LLM), test_llm_tools.py (công cụ Function Calling), test_ml_service.py (tích hợp dịch vụ suy luận), test_performance.py (đo độ trễ P50/P95/P99) và test_edge_cases.py (trường hợp biên); các module dùng chung fixture đăng nhập và cấu hình trong conftest.py.

Endpoint	Method	Status	P95 (ms)	Kết quả
/api/auth/login	POST	200	142	Pass
/api/kpi	GET	200	38	Pass
/api/predictions	GET	200	89	Pass
/api/top-products	GET	200	52	Pass
/api/family-mix	GET	200	41	Pass
/api/family-trend	GET	200	67	Pass
/api/products	GET	200	96	Pass
/api/products?search	GET	200	118	Pass
/api/inventory/restock	POST	200	203	Pass
/api/chat	POST	200	2.340	Pass
Tỉ lệ pass			47/47	

Bảng 4.12: Kết quả kiểm thử API nội bộ (n=50 lần gọi mỗi endpoint).

4.5 Kiểm thử bảo mật

Các tình huống dưới đây được tự động hóa trong module backend/tests/test_security.py của bộ pytest nêu trên.

Tình huống	Kỳ vọng	Thực tế	Kết quả
manager1 truy cập store_nbr=1	200 OK	200 OK	Pass
manager1 truy cập store_nbr=15	403 Forbidden	403 + msg VN	Pass
Token hết hạn	401	401 + redirect login	Pass
Token giả mạo	401	401	Pass
SQL injection qua search	Không thực thi	Tham số hóa	Pass
LLM ngoài phạm vi	Từ chối	validate_tool_access chặn	Pass

Bảng 4.13: Kết quả kiểm thử bảo mật.

4.6 Đánh giá Trợ lý AI (LLM Agent)

4.6.1 Chất lượng hội thoại

Loại câu hỏi	Mức độ	Số test	Tỉ lệ đúng
Phân tích tổng quan cửa hàng	Dễ	10	10/10
Rủi ro cháy hàng cụ thể	Trung bình	10	9/10
Đề xuất nhập hàng SKU	Trung bình	10	9/10
Phân tích hàng tồn chậm	Trung bình	10	10/10
So sánh cụm cửa hàng	Khó	5	4/5
Gợi ý bán chéo	Khó	5	4/5
Câu hỏi ngoài phạm vi	Kiểm biên	5	5/5 từ chối

Bảng 4.14: Kết quả kiểm thử chất lượng LLM Agent trên 55 kịch bản.

4.6.2 Ví dụ hội thoại thực tế

Người dùng: “Cửa hàng 1 có rủi ro cháy hàng không?”

Agent: “Phân tích cửa hàng #1: phát hiện **3 SKU** có rủi ro cháy hàng trong 7 ngày tới:

- item_1503845 (GROCERY I): tồn 12, dự báo 45/ngày $\Rightarrow$ hết trong 0,3 ngày
- item_1047679 (BEVERAGES): tồn 28, dự báo 89/ngày $\Rightarrow$ hết trong 0,3 ngày
- item_1087041 (CLEANING): tồn 5, dự báo 31/ngày $\Rightarrow$ hết trong 0,2 ngày

Thiệt hại ước tính: 67 USD/ngày. Đề xuất: đặt hàng gấp với $Q = \max(0, F_{16d} - \text{Stock})$.”

4.7 Đánh giá hiệu năng tổng thể

Chỉ số	Giá trị	Đánh giá
Thời gian khởi động hệ thống	45 giây	Chấp nhận được
Bộ nhớ ml_service	1.2 GB	Trong ngưỡng
Bộ nhớ backend	280 MB	Tốt
Bộ nhớ frontend nginx	25 MB	Rất nhẹ
Throughput API cache	250 req/s	Đạt yêu cầu
Throughput LLM chat	0.4 req/s	Giới hạn API bên ngoài
Uptime 72 giờ test	99.2%	Đạt yêu cầu

Bảng 4.15: Tổng hợp chỉ số hiệu năng hệ thống.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Tổng kết kết quả đạt được

5.1.1 Kết quả về mô hình học máy

1. Xây dựng thành công **kiến trúc Smart Routing đa cấp**: 33 Local LightGBM chuyên biệt hóa theo ngành hàng + Global Ensemble (LightGBM 41.63% + CatBoost 58.37%);
2. **Kiểm chứng qua Kaggle Leaderboard**: Global Ensemble đạt **NWRMSLE 0.64438 (Public) / 0.63542 (Private — Best Score)**, vượt Local Models (0.65307/0.64207) trên cả hai leaderboard, xác nhận giá trị của chiến lược blend toàn cục;
3. Pipeline feature engineering hoàn chỉnh: 25+ đặc trưng áp dụng nhất quán giữa huấn luyện và suy luận;
4. Dự báo đệ quy 16 ngày trên 59 triệu dòng dữ liệu lịch sử, phân rã tới 4.100 SKU × 54 cửa hàng = 3,37 triệu dòng dự báo.

5.1.2 Kết quả về hệ thống web

1. Triển khai **microservices** 3 dịch vụ trên Docker Compose với healthcheck và tự phục hồi;
2. Xây dựng **Trợ lý AI** tích hợp Groq Cloud với 9 công cụ Function Calling, đạt 90% độ chính xác trên 55 kịch bản kiểm thử;
3. Bảo mật đa tầng: PBKDF2 200k vòng, JWT HS256, RBAC 3 cấp, Row-Level Isolation;
4. Hiệu năng: 100% API endpoint < 1 giây; LLM chat < 3,5 giây (P95).

5.2 Hạn chế của hệ thống

1. **Dữ liệu mô phỏng tồn kho**: Bảng INVENTORY được sinh ngẫu nhiên theo phân phối 60/20/20 — chưa phản ánh tồn kho thực tế;
2. **Khoảng cách với top Kaggle**: NWRMSLE 0.635 so với top 0.35 — do chưa kết hợp Deep Learning, feature engineering còn cơ bản và không dùng multi-seed ensemble;
3. **SQLite hạn chế concurrent write**: WAL mode hỗ trợ đọc song song nhưng ghi vẫn tuần tự;

4. **CORS wildcard:** Cấu hình mở * cho môi trường phát triển — cần siết khi production;
5. **Không có retraining tự động:** Pipeline tách khỏi serving nhưng chưa có scheduler định kỳ;
6. **Đánh giá LLM chủ quan:** 55 kịch bản do nhóm tự thiết kế, chưa có benchmark độc lập.

5.3 Hướng phát triển tương lai

5.3.1 Ngắn hạn (3–6 tháng)

- Chuyển SQLite → PostgreSQL + TimescaleDB;
- Thêm xác thực OAuth2 (Google Workspace);
- WebSocket cho real-time dashboard update;
- Scheduler Airflow retrain model hàng tuần.

5.3.2 Trung hạn (6–12 tháng)

- **Deep Learning:** N-BEATS, TFT (Temporal Fusion Transformer) để thu hẹp khoảng cách với top Kaggle;
- **Probabilistic Forecasting:** Dự báo khoảng tin cậy (quantile regression);
- **Multi-seed ensemble:** Blend 10–20 model với seed khác nhau giảm variance;
- **Reinforcement Learning:** Tối ưu chính sách đặt hàng (s,Q);
- **Multi-tenant isolation:** Schema-per-tenant PostgreSQL.

5.3.3 Dài hạn (1–2 năm)

- Kubernetes (K8s) với horizontal pod autoscaling;
- Tích hợp ERP thực tế (SAP, Odoo);
- Mobile app (React Native);
- Marketplace mô hình: Federated learning.

5.4 Bài học kinh nghiệm

1. **Feature store nhất quán** giữa training và inference là yếu tố quyết định — 80% bug production xuất phát từ schema mismatch;
2. **Global Ensemble thường ổn định hơn Local thuần túy** — kết quả Kaggle (0.635 vs 0.642 private) xác nhận blend toàn cục giảm variance so với 33 local model riêng lẻ;

3. **Private score tốt hơn public là tín hiệu tích cực** — mô hình không overfit, khả năng tổng quát hóa tốt;
4. **LLM Agent cần ràng buộc chặt** — Function Calling với SQL tĩnh giúp tránh 99% hallucination;
5. **Docker healthcheck đáng đầu tư** — 3 dòng YAML tiết kiệm hàng giờ debug.

ĐÁNH GIÁ VÀ NHẬN XÉT

TIÊU CHÍ ĐÁNH GIÁ	ĐIỂM SỐ	NHẬN XÉT VÀ ĐÁNH GIÁ
Ý TƯỞNG ĐỀ TÀI (IDEA)	____ / 10	
TÍNH ỨNG DỤNG (APPLICATION)	____ / 30	
KẾT QUẢ ĐẠT ĐƯỢC (RESULT)	____ / 30	
QUẢN LÝ DỰ ÁN (PROJECT MANAGEMENT)	____ / 10	
BÁO CÁO & THUYẾT TRÌNH (PRESENTATION & REPORT)	____ / 20	
TỔNG ĐIỂM (TOTAL)	____ / 100	

Tài liệu tham khảo

- [1] Corporación Favorita. (2018). *Favorita Grocery Sales Forecasting*. Kaggle. <https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting>
- [2] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 2017.
- [3] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. NeurIPS 2018.
- [4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS 2017.
- [5] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- [6] Qwen Team. (2024). *Qwen2.5 Technical Report*. arXiv preprint arXiv:2412.15115.
- [7] Groq Inc. (2024). *Groq Cloud API Documentation*. <https://console.groq.com/docs>
- [8] Ramírez, S. (2018). *FastAPI Framework*. <https://fastapi.tiangolo.com>
- [9] Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- [10] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). *The M5 Accuracy Competition: Results, Findings, and Conclusions*. International Journal of Forecasting.
- [11] Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). *Inventory and Production Management in Supply Chains* (4th ed.). CRC Press.
- [12] Merkel, D. (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal.
- [13] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519.
- [14] Hipp, D. R. (2000). *SQLite: A Small Fast Embedded SQL Database Engine*. <https://sqlite.org>
- [15] React Team. (2024). *React — The library for web and native user interfaces*. <https://react.dev>
- [16] Bojer, C. S., & Meldgaard, J. (2021). *Kaggle Forecasting Competitions: An Opportunity for Progress*. International Journal of Forecasting, 37(3).

- [17] Makridakis, S., et al. (2022). *M5 competition: Background, organization, and implementation*. International Journal of Forecasting, 38(4).
- [18] Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2019). *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting*. arXiv preprint arXiv:1912.09363.
- [19] Oreshkin, B. N., et al. (2020). *N-BEATS: Neural basis expansion analysis for interpretable time series forecasting*. ICLR 2020.
- [20] Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 29(5).